



TRABAJO FIN DE GRADO
GRADO EN INGENIERÍA INFORMÁTICA
MENCIÓN EN INGENIERÍA DEL SOFTWARE



Diseño y desarrollo de una aplicación de gestión de transportes: ciclo de vida completo del software

Estudiante: Pablo Manzanares López

Dirección: Paula María Castro Castro

A Coruña, agosto de 2026.

Agradecimientos

A mi familia, amistades y profesorado por el apoyo, la paciencia y el acompañamiento durante este trabajo.

Resumen

Este Trabajo Fin de Grado aborda el ciclo de vida completo de una aplicación web para la gestión operativa de una empresa de transportes. El alcance comprende la elicitación y especificación de requisitos a partir de una entrevista con la parte interesada, el diseño del dominio y de la arquitectura, la implementación incremental de backend y frontend, la verificación automatizada, la integración continua y el despliegue a un entorno accesible.

TransitOps combina una API REST desarrollada con ASP.NET Core y PostgreSQL con una aplicación de página única basada en React y TypeScript. El desarrollo siguió una metodología iterativa e incremental en siete incrementos de rebanada vertical, cada uno atravesando diseño, implementación y pruebas para una porción concreta de la especificación. El sistema resultante cubre los catorce requisitos funcionales acordados: acceso con dos roles, catálogos de vehículos, conductores y clientes, gestión de envíos con filtros operativos, asignación de recursos sin duplicidades, ciclo de estados irreversible, historial inmutable de eventos, administración de usuarios e indicadores operativos. Queda desplegado sobre una máquina virtual Linux, accesible por HTTPS mediante un procedimiento documentado y reproducible.

El trabajo no persigue profundidad en una capa concreta, sino disciplina de ingeniería a lo largo de todo el ciclo. Esa disciplina se concreta en cuatro niveles de prueba automatizada elegidos por lo que cada uno puede demostrar, en reglas de negocio garantizadas por el motor relacional y no solo por comprobaciones en servicio, en una trazabilidad explícita desde la entrevista original hasta la evidencia de validación, y en la declaración por escrito de cada limitación aceptada, junto con el incremento de endurecimiento que la resolvió.

Abstract

This Bachelor's Thesis addresses the complete software lifecycle of a web application for the operational management of a transport company. Its scope covers requirements elicitation and specification from a stakeholder interview, domain and architecture design, incremental backend and frontend implementation, automated verification, continuous integration, and deployment to an accessible environment.

TransitOps combines a REST API built with ASP.NET Core and PostgreSQL with a single-page application based on React and TypeScript. Development followed an iterative and incremental method across seven vertical-slice increments, each one crossing design, implementation, and testing for a concrete portion of the specification. The resulting system covers all fourteen agreed functional requirements: two-role access, vehicle, driver, and customer catalogues, shipment management with operational filters, non-duplicated resource assignment,

an irreversible status lifecycle, an immutable event history, user administration, and operational indicators. It is deployed on a Linux virtual machine, reachable over HTTPS through a documented and reproducible procedure.

The work does not pursue depth in any single layer but engineering discipline across the whole lifecycle. That discipline takes concrete form in four levels of automated testing chosen for what each one can demonstrate, in business rules guaranteed by the relational engine rather than by service checks alone, in explicit traceability from the original interview through to validation evidence, and in the written declaration of every accepted limitation alongside the hardening increment that resolved it.

Palabras clave:

- Gestión de transportes
- Ciclo de vida del software
- ASP.NET Core
- React
- Pruebas automatizadas
- Docker

Keywords:

- Transport management
- Software lifecycle
- ASP.NET Core
- React
- Automated testing
- Docker

Índice general

1	Introducción	1
1.1	Contexto del trabajo	1
1.2	Objetivos	2
1.3	Alcance y criterios de éxito	3
1.4	Estructura de la memoria	3
2	Marco tecnológico	4
2.1	Backend y persistencia	4
2.2	Frontend	5
2.3	Pruebas y calidad	7
2.4	Ejecución y automatización	7
3	Metodología y planificación	9
3.1	De la necesidad al incremento	9
3.2	Diseño anticipado e implementación incremental	10
3.3	Planificación temporal	11
3.4	Costes	11
3.5	Definición de hecho	13
3.6	Pruebas y gestión del riesgo	14
4	Análisis y requisitos	15
4.1	Elicitación y especificación	15
4.2	Requisitos funcionales	16
4.3	Reglas de negocio	17
4.4	Casos de uso	17
4.5	Requisitos no funcionales y trazabilidad	21

5	Arquitectura	22
5.1	Organización del backend	23
5.2	Modelo de datos	23
5.3	Evolución del esquema	24
6	Diseño detallado	25
6.1	Autenticación y autorización	25
6.2	Contrato uniforme de API	26
6.3	Sesión en el frontend	27
6.4	Catálogos: baja lógica y unicidad	27
6.5	Envíos: identificación, fechas y consultas	28
6.6	Operación: asignación y ciclo de vida	29
6.7	Trazabilidad: historial inmutable	30
6.8	Administración e indicadores	32
7	Desarrollo iterativo	34
7.1	Sprint 1: cimientos y esqueleto autenticado	34
7.2	Sprint 2: catálogos operativos	35
7.3	Sprint 3: envíos y visibilidad operativa	35
7.4	Sprint 4: asignación y ciclo de vida	38
7.5	Sprint 5: trazabilidad de eventos	41
7.6	Sprint 6: administración e indicadores	44
7.7	Sprint 7: endurecimiento, pruebas de sistema y despliegue	46
8	Validación	48
8.1	Estrategia de pruebas	48
8.2	Evolución de la cobertura por incremento	49
8.3	Validación del sistema completo	52
8.4	Validación sobre el entorno desplegado	54
9	Despliegue y reproducibilidad	56
9.1	Reproducibilidad local	56
9.2	Destino y topología	56
9.3	Entrega continua basada en <i>pull</i>	58
9.4	Procedimiento y automatización	58
9.5	Observabilidad mínima y límites conocidos	60
10	Resultados	61

11 Conclusiones	63
A Trazabilidad de requisitos	66
B Procedimientos de reproducción	68
B.1 Ejecución contenerizada	68
B.2 Verificación automatizada	69
B.3 Pruebas de sistema	69
B.4 Despliegue accesible	70
Lista de acrónimos	71
Glosario	72
Bibliografía	73

Índice de figuras

3.1	Planificación temporal del proyecto en semanas	12
5.1	Arquitectura de integración y ejecución local de TransitOps	22
5.2	Modelo de datos implementado hasta el Sprint 6	23
6.1	Máquina de estados cerrada del envío	30
6.2	Secuencia resumida del Flujo 3: ejecución integral de un envío	31
7.1	Pantalla de login servida por el <i>stack</i> de Docker Compose	35
7.2	Listado de envíos con filtros de estado y fecha persistidos en la URL	37
7.3	Detalle de un envío planificado con cliente y carga estimada	37
7.4	Panel conjunto de asignación en un envío planificado	39
7.5	Asignación guardada con aviso no bloqueante de capacidad	39
7.6	Envío en curso con recogida real y acciones válidas	40
7.7	Envío entregado, con fechas reales y sin acciones de transición	40
7.8	Línea de tiempo con eventos automáticos y manuales del Sprint 5	42
7.9	Formulario de registro de un evento manual	43
7.10	Panel operativo con situación actual y actividad del periodo	45
8.1	Cookie de sesión sobre el despliegue con HTTPS: <code>HttpOnly</code> , <code>Secure</code> y <code>SameSite=Strict</code>	54
8.2	Ejecución de la integración continua en verde tras publicar la rama	55
9.1	Aplicación accesible por la dirección pública del túnel	57
9.2	Ejecución del procedimiento: descarga, salud de los cuatro servicios, revisión y <i>digest</i> desplegados y dirección pública	59
9.3	Temporizador activo, con la ejecución anterior y la siguiente a cinco minutos de distancia	59

9.4	Despliegue completo iniciado por <i>systemd</i> sin intervención, con la misma revisión y los mismos <i>digests</i>	60
-----	---	----

Índice de tablas

3.1	Plan incremental del proyecto	10
3.2	Distribución del esfuerzo por actividad	12
3.3	Estimación de costes del proyecto	13
4.1	Requisitos funcionales	16
4.2	Casos de uso y requisitos que los componen	18
8.1	Pruebas automatizadas del Sprint 1	49
8.2	Validación acumulada de los Sprints 2 y 3	50
8.3	Validación del Sprint 4	51
8.4	Validación del Sprint 5	52
8.5	Validación del Sprint 6	53
A.1	Matriz requisito-sprint-evidencia	66

Introducción

Las pequeñas y medianas empresas de transporte coordinan cada día personas, vehículos, clientes y envíos. Cuando la información queda repartida entre hojas de cálculo, mensajes y llamadas, una pregunta sencilla —qué vehículo está libre o qué ocurrió con una entrega— requiere reconstruir datos de varias fuentes. La fragmentación también favorece errores con impacto directo: asignar el mismo recurso dos veces, emplear datos ya obsoletos o perder la autoría de una incidencia.

TransitOps plantea una aplicación web única para ese trabajo operativo. Permite mantener los catálogos, planificar envíos, asignar recursos, controlar el ciclo de vida y consultar una cronología verificable. Una administración pequeña de usuarios y un resumen de actividad completan el núcleo necesario para que el producto sea demostrable por sí mismo y no una colección de endpoints aislados.

La solución separa una SPA React/TypeScript, una API ASP.NET Core y PostgreSQL, y ofrece ejecución reproducible mediante contenedores. El desarrollo se organiza en rebanadas verticales: primero un esqueleto autenticado y después catálogos, envíos, operación, trazabilidad y administración. Los seis primeros sprints cubren RF-01 a RF-14, y el séptimo añade el endurecimiento, las pruebas de sistema y el despliegue accesible.

1.1 Contexto del trabajo

El trabajo se desarrolla como TFG del Grado en Ingeniería Informática de la Universidad da Coruña, en la mención de Ingeniería del Software. Su objeto es el diseño y la construcción íntegra de una aplicación de gestión operativa para una empresa de transporte, junto con el ciclo de vida de ingeniería que la produce: desde la elicitación de necesidades hasta el despliegue verificado del producto.

La situación de partida combina dos necesidades. Desde el dominio, una empresa de transporte pequeña requiere visibilidad compartida y reglas que eviten inconsistencias. Desde el

TFG, se necesita demostrar competencias de ingeniería más amplias que la programación de una capa: comprender a la parte interesada, delimitar alcance, justificar arquitectura, probar comportamiento, reproducir entornos y evaluar el producto. TransitOps une ambos objetivos en un caso suficientemente rico para contener estados, relaciones, roles y trazabilidad, pero lo bastante pequeño para completarlo.

El interés del proyecto no reside, por tanto, únicamente en implementar funciones. El objetivo académico es aplicar y documentar de forma disciplinada el ciclo de vida completo del software: elicitación, especificación, diseño, implementación, verificación, despliegue y análisis. Cada decisión se relaciona con una necesidad, se materializa en backend y frontend cuando corresponde y se acompaña de pruebas y evidencia. Esta trazabilidad permite valorar no solo qué hace la aplicación, sino cómo se llegó de un problema expresado en lenguaje de negocio a un sistema mantenible.

1.2 Objetivos

El objetivo general es diseñar, construir, probar y desplegar una aplicación web mantenible que permita gestionar las operaciones esenciales de una empresa de transporte y que constituya una demostración verificable del ciclo de vida completo del software. De ese objetivo se derivan los siguientes objetivos específicos:

- Obtener necesidades mediante una entrevista simulada con una responsable del dominio y formalizarlas sin introducir prematuramente decisiones técnicas.
- Mantener trazabilidad entre entrevista, RF-01 a RF-14, RN-01 a RN-17, flujos de negocio, diseño, implementación y pruebas.
- Diseñar un dominio coherente, un contrato HTTP uniforme y una arquitectura separada de backend, frontend y persistencia.
- Implementar incrementos verticales utilizables, con autenticación y autorización desde el comienzo, hasta cubrir el alcance funcional completo.
- Automatizar pruebas de reglas en backend, contratos HTTP y comportamiento observable en frontend.
- Garantizar ejecución local reproducible mediante Docker Compose, configuración externa y migraciones versionadas, sin secretos reales en el repositorio.
- Desplegar el producto en un entorno accesible, ejecutar los cuatro flujos de sistema y analizar los resultados y limitaciones.

- Elaborar documentación técnica y académica que permita reproducir decisiones y defender el proceso seguido.

1.3 Alcance y criterios de éxito

El alcance se mantiene deliberadamente acotado. Incluye usuarios internos con roles de operador y administrador; vehículos, conductores y clientes; envíos con filtros; asignación conjunta; estados; eventos e incidencias; y un resumen operativo. Quedan fuera la optimización de rutas, el seguimiento GPS, la facturación, la aplicación móvil y el acceso de clientes externos. La exclusión no implica que carezcan de valor: evita que capacidades periféricas impidan cerrar y evaluar de extremo a extremo el núcleo elegido. Tampoco se persigue profundidad en infraestructura, porque la integración continua, los contenedores y el despliegue son medios para reproducir y evaluar el producto; la aportación principal es la disciplina aplicada de ingeniería del software a todo su recorrido.

Los criterios funcionales son el cumplimiento verificable de RF-01 a RF-14 y la ejecución satisfactoria de los cuatro flujos de negocio. En calidad, se exige una compilación reproducible, suites automatizadas correctas, esquema migrable desde cero, autorización efectiva en servidor, errores comprensibles y conservación del historial. En operación, la aplicación debe poder levantarse siguiendo documentación explícita y sin incorporar credenciales reales a ficheros versionados.

Los criterios funcionales se alcanzaron durante los seis primeros incrementos con pruebas de componente e integración y verificaciones manuales por rebanada. El séptimo añadió las pruebas [extremo a extremo \(E2E\)](#) de los cuatro flujos, las garantías de concurrencia sostenidas por el motor relacional, una sesión endurecida y el despliegue en un entorno accesible con su evidencia recogida. El éxito solo se declara en ese punto, porque una cobertura amplia y automatizada no equivale a un cierre de sistema demostrado sobre el entorno real.

1.4 Estructura de la memoria

La memoria sigue las fases del ciclo de vida y añade una crónica iterativa. Tras este capítulo lo expone el marco tecnológico, la metodología —con la planificación temporal y la estimación de costes— y los requisitos, y presenta a continuación la arquitectura y el diseño detallado. Los capítulos posteriores conservan la evolución por sprint, la validación, el despliegue, los resultados y las conclusiones. Los anexos cierran la trazabilidad y los procedimientos de reproducción. Esta doble lectura separa el diseño resultante de la evidencia temporal que explica cómo se construyó.

Marco tecnológico

Este capítulo justifica las tecnologías con las que se construyó el sistema. No pretende describirlas —su documentación oficial lo hace mejor— sino explicar por qué se eligió cada una frente a las alternativas disponibles, porque una decisión tecnológica sin alternativa considerada es indistinguible de una costumbre. La selección persigue tres propiedades: coherencia entre componentes, soporte sólido para pruebas y facilidad de reproducción. No se buscó maximizar el número de herramientas: cada incorporación debía resolver un problema que las ya presentes no cubrían, y en varios puntos la decisión defendible resultó ser no añadir nada.

2.1 Backend y persistencia

El backend usa ASP.NET Core sobre .NET 10 [1]. La plataforma reúne servidor [protocolo de transferencia de hipertexto \(HTTP\)](#), inyección de dependencias, configuración, validación, autenticación, autorización y registro. Frente a ensamblar esas piezas sobre un framework mínimo de JavaScript, ofrece convenciones tipadas y un pipeline integrado adecuado para una API con reglas, roles y pruebas. La elección tampoco obliga a una arquitectura empresarial compleja: controladores, servicios y EF Core son suficientes para el tamaño del proyecto.

La plataforma admite también F#, pero C# es su lenguaje por defecto y el que documentan sus bibliotecas, de modo que aquí la decisión se reduce a no apartarse del camino previsto sin una razón de dominio que lo justifique. C# permite expresar contratos mediante tipos, registros y nulabilidad. Las peticiones y respuestas son [objeto de transferencia de datos \(DTOs\)](#) separados de las entidades para no exponer directamente el modelo persistente. La inyección por constructor hace explícitas dependencias como el contexto, el *hasher* o la identidad actual, y facilita sustituirlas en pruebas.

Entity Framework Core actúa como [mapeo objeto-relacional \(ORM\)](#) y administra el esquema mediante [migraciones](#) versionadas [2]. Se prefirió frente a construir [lenguaje de consulta estructurado \(SQL\)](#) manual para cada acceso porque el proyecto necesita evolución incremen-

tal, relaciones y proyecciones repetidas. Ello no elimina el diseño relacional: índices filtrados, restricciones CHECK, tipos temporales y políticas de borrado se configuran explícitamente. Las consultas de lectura emplean `ASNoTracking` y proyecciones para no cargar grafos innecesarios.

PostgreSQL 16 proporciona persistencia relacional, transacciones, claves foráneas e índices parciales [3]. Un almacén documental habría simplificado poco: el dominio contiene relaciones estables y reglas de integridad que encajan de forma natural en un esquema relacional. Los identificadores técnicos son `identificador único universals (UUIDs)`; los identificadores de negocio, como referencia o matrícula, conservan sus propias restricciones. Las fechas que representan instantes se almacenan en `tiempo universal coordinado (UTC)` mediante `timestamp with time zone`.

La elección dentro de lo relacional tampoco resulta indiferente, porque dos reglas del dominio acabaron sostenidas por el motor y no por el servicio: la prohibición de `doble reserva`, que se expresa como `índice único filtrado` único sobre los envíos abiertos, y la protección del último administrador, que necesita serializar un tramo crítico mediante un cerrojo de aviso. MySQL no ofrece índices parciales y habría obligado a simular la condición con una columna derivada; SQLite no admite escritores concurrentes, que es precisamente aquello a lo que esas dos reglas deben resistir; y SQL Server sí dispone de índices filtrados, pero añade una decisión de licencia y una imagen mayor sin aportar nada que el proyecto necesite. PostgreSQL reúne ambas capacidades, cuenta con proveedor EF Core mantenido e imagen oficial, y no impone condiciones de uso.

La autenticación utiliza `JSON Web Token (JWT)`, definido por RFC 7519 [4]. El token firmado transporta identidad y rol, y evita mantener un almacén de sesiones que habría añadido un componente al despliegue y un estado que conservar entre reinicios. La alternativa —sesión en servidor con un identificador opaco en cookie— ofrece revocación inmediata por diseño, y conviene decir que el endurecimiento acabó recuperando esa propiedad sin adoptar el mecanismo: el token viaja en una cookie `HttpOnly` y una versión de sesión comprobada en cada petición lo invalida al instante. La elección final es por tanto un punto intermedio deliberado y no una API sin estado en sentido estricto. ASP.NET Core Identity aporta el `hasher` adaptativo de contraseñas; no se usa el modelo completo de Identity porque dos roles, bootstrap y administración controlada no justifican sus tablas y flujos adicionales.

2.2 Frontend

El frontend es una `aplicación de página única (SPA)` con React 19 [5]. El modelo de componentes permite construir pantallas de catálogo, formularios y detalle operativo compartiendo piezas de presentación sin mezclar reglas del servidor. Se escogió una SPA frente a vistas

renderizadas por la API porque el flujo de asignación, filtros en URL y actualización de cronología se benefician de interacción local; la separación también hace explícito el contrato de integración que el TFG pretende documentar.

Entre las bibliotecas que resuelven ese modelo, la decisión atiende a dos razones concretas. Angular habría traído inyección de dependencias, módulos y programación reactiva propios, una superficie de framework mayor de la que necesitan las pantallas de este producto, y con ella una parte del tiempo dedicada a aprender el marco en lugar del problema. Vue y Svelte lo resolverían igual de bien, pero React tiene detrás la herramienta de prueba con la que este trabajo quería verificar el frontend: consultar la interfaz por rol accesible y texto visible en lugar de por estructura interna. Dicho de otro modo, la biblioteca se eligió por su ecosistema de verificación y no por sus propiedades de renderizado, que para cinco pantallas son equivalentes.

TypeScript añade comprobación estática sobre propiedades, respuestas y estados de interfaz [6]. Su coste de anotación se compensa especialmente en un cliente que consume numerosos DTOs; un cambio de contrato puede detectarse durante la compilación antes de convertirse en una pantalla rota. El frontend no replica las entidades C# automáticamente: mantiene tipos orientados a su uso y conserva la API como frontera.

Vite proporciona servidor de desarrollo, proxy y compilación de producción [7]. Frente a configuraciones manuales de *bundling*, ofrece un punto de partida pequeño y tiempos de realimentación breves. Frente a un armazón con opinión propia sobre el servidor, como los que ofrecen renderizado en servidor y rutas de *backend*, se descartó porque este producto ya tiene su servidor y su contrato: añadir un segundo lugar donde pueda vivir lógica habría difuminado justamente la frontera que la arquitectura pretende hacer visible.

React Router define URLs, parámetros, rutas protegidas y navegación [8]. Los filtros de envíos se representan en la *query string*, lo que aprovecha el navegador como parte del estado de navegación. Se prefirió a resolver el enrutado sobre la API de historial del navegador porque las tres cosas que el producto necesita —rutas cuya autorización depende del rol, un contenedor común de navegación y filtros que viven en la dirección— son exactamente las que la biblioteca ya resuelve, y escribirlas a mano habría añadido código propio que mantener sin comportamiento nuevo que mostrar.

El cliente usa notación de objetos de JavaScript (JSON) sobre HTTP y centraliza token, sobres de éxito y errores. No se incorporó una biblioteca global de estado: contexto para autenticación y estado local por pantalla cubren el alcance actual con menos acoplamiento. Tampoco se añadió un sistema visual externo; CSS propio permite mantener una interfaz coherente y accesible sin depender de componentes cuya personalización excedería el beneficio para este producto.

2.3 Pruebas y calidad

xUnit verifica el backend [9]. Frente a NUnit o MSTest las diferencias funcionales son menores, y pesó una propiedad concreta: xUnit crea una instancia de la clase por cada prueba, lo que encaja con el patrón que esta suite necesita —un contexto de base de datos nuevo y aislado en cada caso— sin recurrir a código de preparación compartido capaz de filtrar estado de una prueba a la siguiente. Las pruebas de servicio ejercitan reglas y persistencia con una infraestructura aislada; las de controlador comprueban políticas, códigos y contrato. Esta separación localiza fallos mejor que probarlo todo exclusivamente a través de HTTP, pero mantiene cobertura del borde real. Las migraciones se validan además contra PostgreSQL en CI, porque un proveedor en memoria no reproduce índices, tipos ni restricciones del motor elegido.

Vitest comparte el modelo de módulos y configuración de Vite [10]; React Testing Library consulta la interfaz por roles, etiquetas y texto visible. Se evita acoplar las pruebas a detalles internos de componentes: una prueba debe observar lo que una persona puede hacer, como filtrar, recibir un aviso o perder acceso a una ruta. Oxlint y la compilación de TypeScript complementan las suites con análisis estático. Se prefirió Oxlint a ESLint, la opción habitual en este ecosistema, porque cubre el conjunto de reglas que este proyecto necesita sin requerir configuración propia y con un tiempo de ejecución que permite invocarlo en cada cierre de sprint sin convertirlo en un trámite que se acabe omitiendo.

2.4 Ejecución y automatización

Docker crea imágenes separadas para API y frontend, y reutiliza la imagen oficial de PostgreSQL [11]. Docker Compose describe tres servicios, dependencias, comprobación de salud, variables y volumen. La alternativa consistía en pedir a quien reproduzca el trabajo que instale .NET, Node y PostgreSQL en su equipo y los configure siguiendo una lista de pasos; se descartó porque haría depender de esa máquina una afirmación central de la memoria, la de que el sistema se levanta desde un documento. Un fichero de composición convierte esa afirmación en una orden única y comprobable, y es también lo que permite que el entorno accesible del Capítulo 9 reutilice la misma descripción con otros parámetros en lugar de mantener una paralela que divergiría.

El contenedor web usa Nginx tanto para servir los estáticos compilados como para reenviar `/api`. Habría sido posible ahorrar ese contenedor sirviendo la SPA desde la propia API, pero entonces el desarrollo —donde el proxy es Vite— y la ejecución contenerizada tendrían fronteras distintas, y el frontend dejaría de poder construirse y publicarse sin reconstruir también el backend. Con Nginx, una demostración local atraviesa la misma frontera que el

desarrollo y que el despliegue. La configuración sensible se obtiene de un `.env` ignorado y el Compose falla de forma explícita si falta una variable obligatoria.

GitHub Actions ejecuta restauración, *lint*, compilación, pruebas y validación de migraciones [12]. Se eligió por proximidad y no por preferencia: el repositorio ya está alojado en GitHub, de modo que la integración no añade ninguna cuenta, ningún servicio externo ni ningún secreto que custodiar, y su cuota gratuita para repositorios públicos cubre el uso del proyecto. Un servidor propio, del tipo Jenkins, habría exigido operarlo, que es precisamente la actividad que este trabajo declara fuera de su alcance; y una plataforma de terceros habría introducido una credencial de acceso al código a cambio de capacidades que aquí no se emplean. El mismo argumento explica que el registro de contenedores del que se sirve el despliegue sea el de la propia cuenta.

La automatización no constituye un despliegue complejo: es una red de seguridad que demuestra que el repositorio puede reconstruirse fuera del equipo de desarrollo. Esta realimentación frecuente se alinea con la entrega continua [13]. El destino del despliegue se decidió deliberadamente en el séptimo incremento y no antes: escogerlo sin conocer las necesidades reales del producto habría supuesto anteponer la infraestructura a lo que debe sostener.

Vista en conjunto, la selección comparte una propiedad que la estimación de costes recoge más adelante: ninguna de estas herramientas exige licencia ni cuota para este uso. No fue una restricción impuesta desde fuera, sino un criterio de la propia elección, porque un trabajo que solo pudiera reproducirse pagando dejaría de ser reproducible para quien lo lee.

Metodología y planificación

Este capítulo explica cómo se organizó el trabajo: qué proceso de desarrollo se siguió, cómo se convirtió una entrevista en una secuencia de incrementos verificables, cuánto tiempo se dedicó a cada actividad y qué coste tendría producir el sistema en un contexto profesional. Se adopta un proceso iterativo e incremental adaptado a un proyecto individual. La ingeniería del software comprende actividades relacionadas —especificación, desarrollo, validación y evolución— que no necesitan ejecutarse una única vez como fases estancas [14]. En Transi-tOps cada incremento recorre esas actividades para una capacidad acotada y deja una versión integrada.

3.1 De la necesidad al incremento

La entrevista simulada con una responsable de operaciones constituye la fuente primaria. Sus respuestas se transforman en requisitos funcionales, reglas de negocio, requisitos no funcionales y cuatro flujos. El *roadmap* agrupa después ese material en ocho sprints. La cadena entrevista → requisito → diseño → prueba conserva la razón de cada decisión y evita que el *backlog* sea una lista de ideas técnicas sin necesidad asociada.

La unidad de planificación es la *rebanada vertical*. Por ejemplo, “gestionar vehículos” incluye entidad, migración, servicio, endpoint, pantallas y pruebas; no se planifica como una fase de todas las tablas seguida meses después por todas las interfaces. Una estrategia horizontal habría producido mucho código no demostrable hasta el final y habría retrasado el descubrimiento de incompatibilidades entre navegador, API y PostgreSQL. Las rebanadas pequeñas entregan evidencia y permiten reajustar la siguiente decisión.

El Sprint 1 usa un *esqueleto andante*: login y ruta protegida cruzan configuración, base de datos, API, proxy y SPA. Esa base no pretende anticipar toda la funcionalidad, sino retirar pronto los riesgos transversales. Los Sprints 2 a 6 añaden catálogos, envíos, operación, eventos y administración. El Sprint 7 reúne endurecimiento, pruebas de sistema y despliegue; el Sprint

8 cierra documentación y defensa.

La Tabla 3.1 resume ese reparto asociando cada sprint con los requisitos que lo justifican. Conviene leerla como una asignación de responsabilidad y no como un cronograma: su valor está en que ningún sprint aparece sin requisitos que lo motiven y ningún requisito queda sin sprint asignado. Los dos últimos incrementos se salen del patrón deliberadamente, porque no añaden funcionalidad nueva: el séptimo consolida cualidades transversales sobre lo ya construido y el octavo cierra la documentación.

Tabla 3.1: Plan incremental del proyecto

Sprint	Incremento	Requisitos principales
1	Esqueleto autenticado	RF-01, RF-02, RF-13
2	Catálogos	RF-05, RF-06, RF-07
3	Envíos y filtros	RF-08, RF-12
4	Asignación y ciclo de vida	RF-09, RF-10
5	Historial de eventos	RF-11
6	Usuarios, contraseña y resumen	RF-03, RF-04, RF-14
7	Sistema completo y despliegue	RNF y E2E
8	Documentación y defensa	Cierre

3.2 Diseño anticipado e implementación incremental

En el Sprint 1 se diseñó el modelo conceptual completo. Esta visión reduce el riesgo de que cada rebanada invente nombres, cardinalidades o políticas de borrado incompatibles. Sin embargo, solo se migró `AppUser`; cada tabla posterior apareció cuando existían reglas, end-points y pruebas que la utilizaban. El equilibrio es deliberado: arquitectura suficiente para orientar, implementación justa para demostrar.

Las decisiones se registran en artefactos versionados. `docs/design/` conserva el modelo y la integración; cada `SprintNPlan.md` fija decisiones antes de implementar; `CONTEXT.md` mantiene un *log* fechado; y `docs/Roadmap.md` preserva cierres y evidencia. El código implementado prevalece sobre un plan si aparece una diferencia, y la documentación se corrige para describir el resultado real.

3.3 Planificación temporal

La dedicación se dimensionó a partir de los créditos asignados al Trabajo Fin de Grado: doce créditos ECTS que, a razón de veinticinco horas por crédito, suponen trescientas horas de trabajo. Distribuidas en diez semanas efectivas, resultan unas treinta horas semanales. Esa cifra no es una medición a posteriori, sino la restricción de partida: el alcance se acotó para caber en ella, y ahí reside una de las decisiones de ingeniería del proyecto, porque un alcance mayor habría obligado a renunciar a las pruebas o al despliegue.

La unidad de calendario es la semana y coincide con la duración de un sprint. La equivalencia es intencionada: una rebanada vertical que no cabe en una semana suele abarcar más de una capacidad y conviene dividirla. Las dos primeras semanas no producen incremento ejecutable porque se dedican a entender el problema y a fijar el modelo conceptual; a partir de la tercera, cada semana cierra con una versión integrada y demostrable.

La Figura 3.1 representa ese reparto. Las ocho barras centrales son los sprints y avanzan en secuencia, sin solapamiento, porque un único desarrollador no puede paralelizarlos. Las dos barras inferiores, en cambio, se extienden a lo largo de todo el desarrollo: la verificación automatizada arranca con el primer sprint y crece con cada incremento, en lugar de concentrarse en una fase final de pruebas, y la redacción de la memoria acompaña al proyecto desde la primera semana para que cada capítulo se escriba con la decisión reciente y no reconstruida meses después. Esa continuidad es la razón de que no exista una fase de documentación al final del diagrama.

El reparto de esas trescientas horas entre actividades aparece en la Tabla 3.2. La implementación concentra algo menos de la mitad del esfuerzo, lo que resulta coherente con la tesis del trabajo: si escribir código hubiese absorbido la mayor parte del tiempo, las actividades que este TFG pretende demostrar —especificación, verificación y despliegue— habrían quedado como apéndices de la programación en lugar de como partes equiparables del ciclo de vida. La verificación recibe más horas que el diseño, proporción deliberada en un proyecto cuyo criterio de cierre exige pruebas automatizadas por incremento, y sumada a la especificación y al despliegue supera a cualquiera de las dos fases de implementación por separado.

3.4 Costes

El proyecto no tuvo coste monetario para su autor, pero estimar el que tendría en un contexto profesional es parte del ejercicio de ingeniería: permite comprobar si las decisiones técnicas son proporcionadas al valor del producto. La Tabla 3.3 recoge esa estimación.

El personal domina el resultado, como es esperable en un proyecto de software sin inversión en infraestructura. Se valora a razón de veinte euros por hora, tarifa correspondiente al

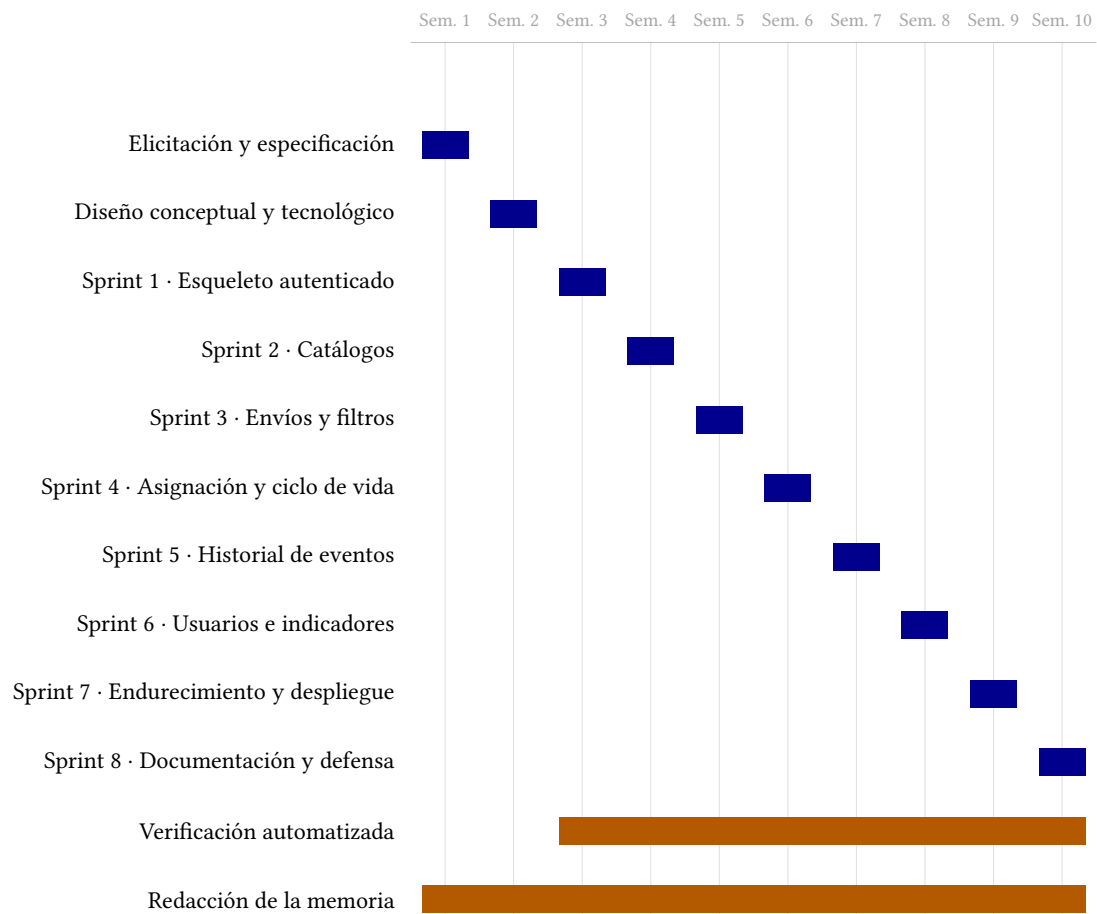


Figura 3.1: Planificación temporal del proyecto en semanas

Tabla 3.2: Distribución del esfuerzo por actividad

Actividad	Horas	%
Elicitación y especificación de requisitos	30	10
Diseño de arquitectura y diseño detallado	40	13
Implementación del backend	80	27
Implementación del frontend	60	20
Verificación automatizada	45	15
Despliegue y automatización de la entrega	20	7
Redacción de la memoria	25	8
Total	300	100

coste para la empresa de un ingeniero *junior*, aplicada a las trescientas horas planificadas. El equipo de desarrollo es un portátil de 1.200 euros amortizado a cuatro años, del que el proyecto consume dos meses y medio. Los costes indirectos de energía y conectividad se estiman de forma agregada, sin desglosarlos, porque una precisión mayor sería falsa.

Tabla 3.3: Estimación de costes del proyecto

Concepto	Base de cálculo	Importe
Personal	$300 \text{ h} \times 20 \text{ €/h}$	6.000,00 €
Amortización del equipo	$1.200 \text{ €} \times 2,5/48 \text{ meses}$	62,50 €
Licencias de software	Herramientas libres o gratuitas	0,00 €
Infraestructura y servicios	Máquina local, CI y registro gratuitos	0,00 €
Costes indirectos	Energía y conectividad, estimados	75,00 €
Total		6.137,50 €

Las dos líneas a cero merecen comentario, porque no son un descuido de la estimación sino una consecuencia de decisiones tomadas y justificadas en su momento. Ninguna herramienta del proyecto exige licencia: la plataforma de desarrollo, el motor de base de datos, las bibliotecas de frontend, los ejecutores de pruebas y la contenerización son libres o gratuitos para este uso. Y la infraestructura tampoco cuesta: la integración continua se ejecuta dentro de la cuota gratuita para repositorios públicos, el registro de contenedores es gratuito en las mismas condiciones, el túnel que publica la aplicación no requiere cuenta y la máquina de despliegue es virtual sobre el equipo ya contabilizado.

De ahí se sigue una propiedad que interesa a RNF-06: el coste marginal de mantener el sistema accesible es nulo. Una plataforma gestionada habría añadido una cuota mensual recurrente sin aportar nada a la tesis de este trabajo, que es el ciclo de vida del software y no la operación de infraestructura. La decisión de desplegar sobre una máquina propia, tomada en el séptimo incremento, queda así respaldada también por el análisis económico y no solo por la comodidad.

3.5 Definición de hecho

Un sprint no se considera terminado cuando el código “parece funcionar”. Su definición de hecho exige:

- requisitos y decisiones de alcance explícitos;
- reglas situadas en la capa responsable y contrato HTTP coherente;

- flujo de frontend completo cuando la capacidad es visible;
- migración reproducible si cambia el esquema;
- pruebas de comportamiento nuevas o actualizadas;
- *lint*, compilaciones y suites en verde;
- verificación integrada proporcional al riesgo, incluida PostgreSQL real cuando importa;
- actualización de *roadmap*, contexto, diseño y memoria con evidencia del incremento.

Esta lista aproxima una política de calidad repetible y evita cierres basados solo en percepción. La CI aporta una comprobación independiente, pero no sustituye las decisiones ni la inspección del flujo real. Capturas, recuentos de pruebas y verificaciones Docker registran lo observado en el momento del cierre.

3.6 Pruebas y gestión del riesgo

La estrategia combina pruebas rápidas de servicios, pruebas de controlador y pruebas de interfaz orientadas a comportamiento. Las reglas críticas se ejercitan donde viven; los contratos se verifican en el borde; y los recorridos completos se reservan para integración y Sprint 7. Esta pirámide evita depender exclusivamente de pruebas E2E lentas sin renunciar a validar el sistema ensamblado.

Cada sprint reduce primero su riesgo dominante. En catálogos fue la conservación tras baja; en envíos, fechas UTC y filtros; en operación, estados y reservas; en eventos, atomicidad y autoría; en administración, el último administrador. Las limitaciones no resueltas se registran, no se ocultan: la concurrencia en RN-04 y RN-12, el token no revocable y el almacenamiento del token en el navegador formaron entradas concretas del endurecimiento posterior.

No se aplican ceremonias de equipo que no aportan a un proyecto individual. La planificación, la revisión de alcance, la demostración mediante evidencia y la retrospectiva documental sí se mantienen, coherentes con la idea de adaptar el proceso al tamaño y a las personas [15]. El resultado es una metodología ligera, pero auditable.

Análisis y requisitos

Este capítulo recoge qué debe hacer el sistema y de dónde procede cada exigencia. Se organiza siguiendo el recorrido que hizo la información: primero cómo se obtuvo de la parte interesada y cómo se convirtió en una especificación comprobable; después los requisitos funcionales, las reglas de negocio que los hacen inequívocos y los casos de uso que los encadenan en tareas reales; y por último los requisitos no funcionales y el mecanismo de trazabilidad que permite auditar el conjunto. El orden importa porque sostiene una afirmación que el resto de la memoria da por buena: ningún requisito de este capítulo se inventó durante la implementación.

4.1 Elicitación y especificación

La fuente inicial es una entrevista simulada con Marta Souto, responsable de operaciones de una pyme ficticia de transporte. El formato conversacional obliga a partir de problemas y lenguaje del dominio: información dispersa, recursos duplicados, falta de historial y necesidad de una herramienta sencilla. No se preguntó qué framework o base de datos quería la cliente, porque esas son decisiones de solución.

Las respuestas se consolidaron en `docs/ClientRequirements.md`. A partir de ellas se redactó `docs/Requirements.md` como especificación canónica no técnica. Cada requisito conserva un campo *Origen* que apunta a la parte de la entrevista que lo justifica. Una revisión de coherencia añadió aspectos que la conversación exigía pero una primera redacción no hacía comprobables: carga estimada para que la capacidad tuviese uso, prevención de doble reserva, primer administrador, clientes y cambio de contraseña. La entrevista y la especificación se actualizaron juntas, sin inventar unicidades no solicitadas.

Los actores son operador, administrador, público no identificado e instalador. Conductores y clientes son información del dominio, pero RN-16 aclara que no acceden a la aplicación. Esta distinción evita convertir por analogía cada persona registrada en una cuenta.

4.2 Requisitos funcionales

La Tabla 4.1 enumera los catorce requisitos funcionales acordados con su prioridad. La columna de prioridad no expresa importancia sino orden de construcción: los marcados como altos forman la ruta crítica sin la cual el producto no es demostrable, mientras que los medios completan un sistema utilizable. Conviene notar que ningún requisito quedó clasificado como bajo, porque los candidatos de esa categoría —optimización de rutas, seguimiento por GPS, facturación— se excluyeron del alcance en lugar de mantenerse en una lista que nunca se abordaría.

Tabla 4.1: Requisitos funcionales

ID	Descripción	Prioridad
RF-01	Acceso con usuario y contraseña y permisos por rol.	Alta
RF-02	Creación controlada del primer administrador.	Alta
RF-03	Cambio de contraseña propia confirmando la actual.	Media
RF-04	Administración de usuarios, roles, activación y asignación de contraseña.	Media
RF-05	Gestión y baja lógica de vehículos.	Alta
RF-06	Gestión y baja lógica de conductores.	Alta
RF-07	Gestión y baja lógica de clientes.	Media
RF-08	Creación, consulta y edición de envíos.	Alta
RF-09	Asignación no duplicada de vehículo y conductor.	Alta
RF-10	Estados planificado, en curso, entregado y cancelado.	Alta
RF-11	Historial trazable de eventos del envío.	Alta
RF-12	Visibilidad y filtros operativos de envíos.	Alta
RF-13	Validación y mensajes comprensibles.	Alta

RF-14

Resumen estadístico operativo.

Media

Las prioridades orientan el orden, no eliminan alcance. Autenticación, operación básica, trazabilidad y claridad de errores forman la ruta crítica; clientes, autoservicio de contraseña, usuarios e indicadores completan después un producto utilizable. Al cierre del Sprint 6 todos los RF quedaron integrados, y la validación de sistema global llegó con el séptimo.

4.3 Reglas de negocio

Las diecisiete RN concretan condiciones que una descripción funcional breve no debe dejar ambiguas. Se agrupan en cuatro áreas:

- **Asignación y recursos (RN-01 a RN-05):** un envío mantiene una pareja vehículo–conductor; solo se modifica planificado; cliente y recursos deben estar activos; no se permite **double reserva**; y una capacidad insuficiente avisa sin bloquear.
- **Fechas, estados y traza (RN-06 a RN-09):** las fechas previstas son coherentes; iniciar requiere asignación; los estados finales no revierten; y cada evento pertenece a un envío y conserva autoría.
- **Acceso y administración (RN-10 a RN-14, RN-17):** solo el administrador gestiona cuentas; el bootstrap se limita al primero; siempre queda un administrador activo; un usuario inactivo no entra; el cambio de contraseña propia exige confirmar la actual; y un administrador puede asignar una contraseña nueva a otra persona sin conocer la anterior, lo que cierra sus sesiones abiertas y no es aplicable a su propia cuenta.
- **Conservación y actores (RN-15 y RN-16):** las bajas de catálogo no borran historia, y conductores y clientes no son usuarios.

Las reglas actúan como puente entre lenguaje de negocio y criterios de prueba. Por ejemplo, RF-09 dice asignar recursos, mientras RN-02, RN-03, RN-04 y RN-05 determinan exactamente cuándo se acepta, rechaza o advierte. Esta separación reduce frases excesivamente largas y permite citar la misma regla desde diseño, servicio y matriz de trazabilidad.

4.4 Casos de uso

Los requisitos funcionales describen capacidades aisladas, pero una aplicación puede satisfacer cada una por separado y resultar inservible al encadenarlas. Por eso la especificación define además cuatro casos de uso que recorren tareas completas del negocio. No introducen

requisitos nuevos: cada uno se compone de RF y RN ya enumerados, y su valor está en fijar el orden, las precondiciones y lo que debe quedar cierto al terminar. La Tabla 4.2 los resume antes de detallarlos.

Tabla 4.2: Casos de uso y requisitos que los componen

ID	Objetivo	Actor principal	Requisitos
CU-1	Poner en marcha una instalación vacía	Instalador	RF-02, RN-11
CU-2	Incorporar a una persona al sistema	Administrador, operador	RF-01, RF-03, RF-04
CU-3	Ejecutar un envío de principio a fin	Operador	RF-05–12
CU-4	Retirar el acceso sin perder historial	Administrador	RF-04, RN-12, RN-15

CU-1: puesta en marcha de una instalación vacía

El actor es el instalador, único punto del sistema donde alguien opera sin haber iniciado sesión, porque todavía no existe ninguna cuenta con la que hacerlo. La precondición es una base de datos migrada y sin usuarios, y el conocimiento del secreto de instalación que se configura fuera del código.

1. El instalador envía usuario, correo y contraseña al punto final de arranque, acompañados del secreto de instalación en una cabecera.
2. El sistema comprueba que no existe ya un administrador activo (RN-11) y que el usuario y el correo no están en uso.
3. El sistema crea la cuenta con rol de administrador y almacena la contraseña transformada por el componente adaptativo de *hash*.
4. El administrador accede con esas credenciales y obtiene una sesión válida.

La excepción relevante es el segundo intento: una vez creada la cuenta, cualquier invocación posterior se rechaza con un conflicto, incluso presentando el secreto correcto. Esa irreversibilidad es intencionada y convierte el arranque en una operación de instalación y no en una vía alternativa de creación de administradores. Un secreto incorrecto se rechaza sin revelar si la instalación ya estaba inicializada. Al terminar, el sistema tiene exactamente un administrador activo y el punto final queda inservible.

CU-2: incorporación de una persona al sistema

Intervienen dos actores en momentos distintos: un administrador autenticado que crea la cuenta y el operador que la recibe. La precondition es la existencia de un administrador activo, es decir, que CU-1 se haya completado.

1. El administrador crea una cuenta indicando usuario, correo, rol y una contraseña inicial.
2. El sistema valida la unicidad del usuario y del correo frente a todas las cuentas, incluidas las inactivas.
3. El operador accede con la contraseña inicial y el sistema le concede una sesión con los permisos de su rol.
4. El operador cambia su contraseña confirmando la actual (RN-14).
5. El sistema invalida las sesiones abiertas de esa cuenta, de modo que el cambio surte efecto de inmediato.

Las excepciones cubren tres situaciones: un usuario o correo repetido se rechaza con un conflicto que identifica el campo en cuestión; una contraseña actual incorrecta impide el cambio; y una contraseña nueva idéntica a la vigente también, porque rotarla por sí misma no cumple ninguna finalidad. Si el operador olvida la credencial, la vía no es este caso de uso sino la asignación por parte de un administrador que introdujo RN-17. Al terminar, existe una cuenta operativa cuya contraseña solo conoce su titular.

CU-3: ejecución de un envío de principio a fin

Es el caso de uso central y el que justifica la mayor parte del sistema. El actor es un operador autenticado y la precondition es disponer de los catálogos que el envío necesita. La Figura 6.2 representa la secuencia técnica de su tramo principal.

1. El operador mantiene los catálogos: registra o consulta el vehículo, el conductor y el cliente que intervendrán.
2. Crea el envío con su referencia, el cliente, los puntos de origen y destino, las fechas previstas y la carga estimada.
3. Asigna vehículo y conductor como una pareja indivisible, que el sistema acepta solo si ambos están activos y libres en los estados abiertos (RN-03, RN-04).
4. Si la carga supera la capacidad del vehículo, el sistema avisa pero no impide la asignación (RN-05).

5. Inicia el envío, lo que exige asignación previa (RN-07) y sella la hora real de recogida.
6. Registra durante el trayecto los puntos de control o las incidencias que procedan, con la hora real que el operador indique.
7. Marca el envío como entregado, sellando la hora real de entrega, o lo cancela.

Las excepciones son las que más valor aportan al caso de uso, porque son las que un sistema mal construido acepta. Asignar un recurso ya ocupado en otro envío abierto se rechaza identificando cuál lo ocupa. Modificar la asignación de un envío que ya no está planificado se rechaza (RN-02). Una fecha de entrega anterior a la de recogida se rechaza en el contrato, en el servicio y en la base de datos. Y una transición desde un estado final no se admite en ningún caso (RN-08), ni siquiera para corregir un error, porque el historial es la vía prevista para dejar constancia de una rectificación. Al terminar, el envío se encuentra en un estado terminal y su cronología permite reconstruir qué ocurrió, cuándo y por decisión de quién.

CU-4: retirada del acceso conservando el historial

El actor es un administrador y la precondition es la existencia de una cuenta activa distinta de la suya, junto con actividad previa atribuida a esa cuenta.

1. El administrador desactiva la cuenta seleccionada.
2. El sistema comprueba que la operación no deja el sistema sin ningún administrador activo (RN-12).
3. El sistema invalida las sesiones abiertas de la cuenta desactivada.
4. Un intento de acceso posterior con esas credenciales se rechaza con el mismo mensaje genérico que una contraseña incorrecta.
5. Los eventos registrados por esa persona siguen mostrando su autoría en la cronología de los envíos afectados (RN-15).

La excepción crítica es desactivar al último administrador activo, que el sistema rechaza con un conflicto explicativo; el caso simétrico, degradar su rol, se rechaza por la misma regla. Al terminar, la persona ha perdido el acceso sin que el sistema haya perdido la trazabilidad de lo que hizo, que es precisamente la razón de que las bajas sean lógicas y no borrados.

Estos cuatro casos de uso son también los recorridos que el séptimo incremento automatizó con pruebas de sistema. El resto de la memoria los denomina indistintamente CU-*n* o Flujo *n*: el segundo nombre es el que conservan las pruebas de Playwright en el repositorio, y mantenerlo permite que una prueba fallida señale sin ambigüedad qué caso de uso ha dejado de cumplirse. Especificación y validación comparten así unidad de descripción.

4.5 Requisitos no funcionales y trazabilidad

RNF-01 exige facilidad de uso desde navegador; RNF-02, acceso seguro; RNF-03, fiabilidad y conservación; RNF-04, trazabilidad de eventos; RNF-05, disponibilidad compatible con la operativa; y RNF-06, rendimiento adecuado para una empresa con decenas de recursos. Son deliberadamente proporcionales al contexto: no se atribuyen objetivos de escala global a una pyme ficticia, pero tampoco se confunde “pequeño” con ausencia de seguridad o pruebas.

La matriz del Apéndice A enlaza grupos de requisitos con sprints, componentes y evidencia, y el Apéndice B recoge los procedimientos que permiten reproducir esa evidencia sin conocimiento no escrito. Los planes de sprint amplían criterios; las suites los automatizan; y el *roadmap* registra el cierre. Los cuatro casos de uso quedaron automatizados en el séptimo incremento, y la comprobación de RNF-05 se realizó sobre el entorno desplegado que describe el Capítulo 9, de modo que ningún requisito no funcional queda respaldado únicamente por una afirmación de diseño.

Arquitectura

La solución aplica una separación cliente-servidor entre una SPA, una [interfaz de programación de aplicaciones \(API\)](#) REST y PostgreSQL. El navegador nunca accede directamente a la persistencia: autenticación, autorización, validación y reglas de negocio se evalúan en la API. Esta frontera permite que la interfaz se concentre en interacción y presentación sin convertirse en una segunda fuente de reglas.

La Figura 5.1 representa tanto la integración lógica como la topología local reproducible. Nginx sirve los recursos estáticos generados por Vite y reenvía las solicitudes bajo `/api` al contenedor ASP.NET Core. La API accede a PostgreSQL mediante EF Core y la base de datos conserva sus archivos en un volumen nombrado. Durante el desarrollo sin contenedores Vite sustituye a Nginx como proxy, pero mantiene el mismo contrato [HTTP](#).

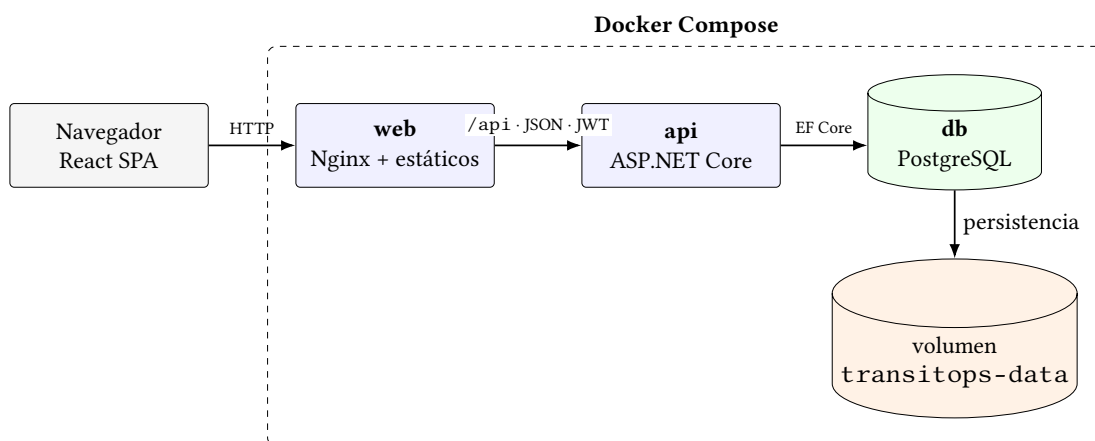


Figura 5.1: Arquitectura de integración y ejecución local de TransitOps

5.1 Organización del backend

El backend combina una organización por responsabilidad transversal con módulos por característica. `Domain/` contiene las entidades y enumeraciones sin depender de transporte HTTP; `Features/` agrupa contratos y servicios por capacidad —autenticación, catálogos, envíos, usuarios e indicadores—; y `Controllers/` publica esas capacidades como recursos y acciones REST. Esta organización por característica mantiene próximos el contrato y la lógica que cambian juntos, sin introducir una jerarquía de capas artificialmente profunda.

Las preocupaciones compartidas tienen límites visibles. `Persistence/` contiene el contexto EF Core, la factoría de diseño y las *migraciones*; `Security/` configura roles, tokens y la identidad actual; `Middleware/` unifica errores, correlación y registro; y `Common/` reúne tipos de resultado y excepciones de aplicación. Los controladores permanecen delgados: traducen el protocolo y delegan, mientras que las reglas comprobables residen en servicios. Esta división permite probar las decisiones de negocio sin servidor y reservar las pruebas de controlador para políticas, códigos y forma de las respuestas.

5.2 Modelo de datos

El modelo completo se diseñó antes de implementar las rebanadas funcionales, pero se materializó de forma incremental. La Figura 5.2 recoge las seis entidades actuales, sus claves principales y las cardinalidades relevantes. `Shipment` es el *agregado* operativo central: referencia opcionalmente cliente, vehículo y conductor. `ShipmentEvent` pertenece obligatoriamente a un envío y puede conservar la identidad del usuario autor.

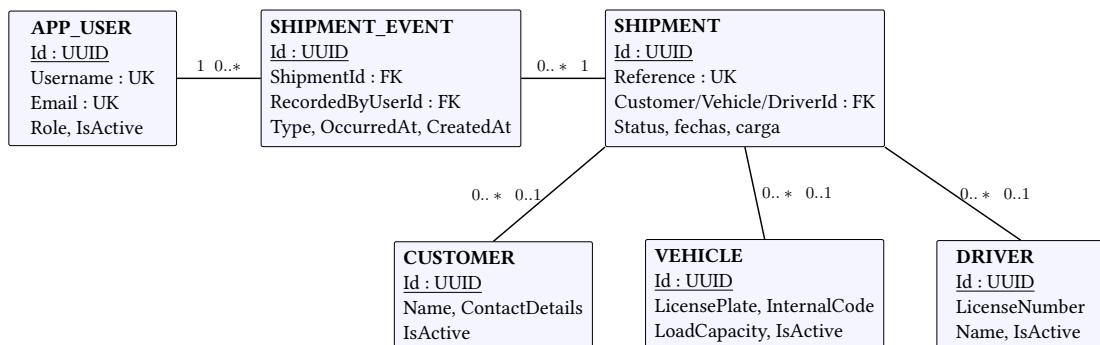


Figura 5.2: Modelo de datos implementado hasta el Sprint 6

Usuarios y catálogos emplean *baja lógica*; los envíos expresan su vigencia mediante una *máquina de estados*. Las claves foráneas desde el envío usan borrado restrictivo para conservar referencias históricas incluso si un recurso se desactiva. La relación envío–evento usa cascada como excepción deliberada: el evento no tiene sentido fuera del agregado, aunque la API

actual tampoco ofrece borrado de envíos. Los nombres, atributos, índices y restricciones se documentan con mayor detalle en `docs/design/DataModel.md`.

5.3 Evolución del esquema

El esquema activo resulta de cinco migraciones EF Core ordenadas: `InitialCreate` crea `app_users`; `AddCatalogTables` incorpora vehículos, conductores y clientes; `AddShipments` añade el agregado y sus relaciones; `AddShipmentOperation` agrega asignación, estados y fechas reales; y `AddShipmentEvents` materializa la traza inmutable. El Sprint 6 no necesitó una nueva migración porque la administración reutiliza `AppUser` y el resumen calcula agregados sobre datos existentes.

Esta secuencia hace reproducible la evolución y mantiene cada incremento desplegable. Diseñar el modelo completo desde el inicio evitó contradicciones entre rebanadas; aplazar la creación física de tablas hasta que existía comportamiento que las utilizase evitó, a su vez, un esquema amplio sin pruebas ni interfaz asociada.

Diseño detallado

Este capítulo describe las decisiones que forman el sistema tras los siete incrementos de desarrollo. A diferencia de la crónica del Capítulo 7, el orden no es temporal: cada sección presenta una capacidad, las alternativas consideradas y las invariantes que conserva.

Las secciones siguen el recorrido de una petición por el sistema, de fuera hacia dentro: primero cómo se establece y se comprueba la identidad, después la forma común de las respuestas y la sesión en el cliente, y a continuación las capacidades de dominio —catálogos, envíos, operación, trazabilidad y administración— en el orden en que dependen unas de otras. Cada una enuncia también lo que se descartó, porque una decisión de diseño sin su alternativa no se puede evaluar: informa de qué se hizo, pero no de si era la mejor opción disponible. Cuando una regla se sostiene en más de una capa, el capítulo indica cuál aporta la corrección y cuál el mensaje comprensible, distinción que reaparece en varias secciones y que no es redundancia sino reparto deliberado de responsabilidades.

6.1 Autenticación y autorización

El arranque controlado de RF-02 resuelve el problema circular de una instalación vacía: todavía no existe un administrador que pueda crear al primero. `POST /api/v1/auth/bootstrap-admin` recibe un secreto de instalación mediante la cabecera `X-Bootstrap-Token`, configurado fuera del código. Antes de crear la cuenta comprueba RN-11 —que no exista ya un administrador activo— y la unicidad global de usuario y correo. Se eligió un endpoint de uso único frente a credenciales sembradas porque no introduce una contraseña conocida en la imagen ni obliga a manipular directamente la base de datos.

Las contraseñas se transforman con el componente adaptativo de ASP.NET Core Identity. El login aplica deliberadamente el mismo error a usuario inexistente, contraseña incorrecta y usuario inactivo para no revelar qué cuentas están registradas. Un acceso válido emite un [JWT](#)

firmado que contiene sub, nombre, correo, rol, identificador del token y caducidad. Firma, emisor, audiencia y tiempo de vida se validan en cada petición protegida.

Las políticas *Operational* y *Admin* centralizan RN-01 y RN-10. La primera admite ambos roles internos; la segunda solo administradores. La SPA adapta rutas y navegación al rol para evitar acciones inútiles, pero esa visibilidad es una ayuda de uso, no un control de seguridad: los controladores vuelven a exigir la política correspondiente. Esta duplicidad aparente separa correctamente experiencia y autoridad.

Hasta el Sprint 6 el token fue puramente autocontenido: no se consultaba la cuenta en cada solicitud, de modo que una desactivación o un cambio de rol no invalidaba los claims ya emitidos hasta que el JWT caducaba. La limitación se declaró como deuda explícita y el endurecimiento la resolvió con una versión de sesión. La cuenta almacena un contador `TokenVersion` que viaja como claim adicional; al validar el token, un evento del manejador JWT resuelve el usuario, comprueba que siga activo y compara la versión. Cambiar la contraseña, desactivar la cuenta o modificar el rol incrementan el contador, así que los tokens anteriores dejan de servir en la siguiente petición.

La misma consulta cubre los tres casos sin código separado para cada uno: si el usuario ha sido desactivado no aparece en el filtro y la comparación falla igualmente. Se descartaron tanto una lista de revocación como un esquema de *refresh tokens*: ambos añaden estado y endpoints que proteger, mientras que para el volumen de RNF-06 una lectura ligera de usuario por petición ofrece invalidación inmediata a un coste medible y acotado. El rechazo reutiliza el mismo contrato 401 que el resto de la autenticación.

6.2 Contrato uniforme de API

RF-13 exige que un fallo sea comprensible para quien opera y tratable de forma estable por el frontend. Las respuestas correctas envuelven el resultado en `data` y añaden `requestId`. Los errores incluyen `error.code`, `error.message`, detalles opcionales y el mismo identificador de correlación. El código es estable para la interfaz; el mensaje se presenta a la persona; el identificador enlaza la incidencia con los registros del servidor.

Un middleware común traduce excepciones de aplicación y fallos inesperados. Los eventos del manejador JWT completan la misma forma para 401 y 403, que de otro modo quedarían fuera del flujo MVC. La API mantiene así un contrato único para validación (400), ausencia (404), conflicto de reglas (409), autenticación, autorización y error interno. Se descartó devolver directamente excepciones o formatos distintos por controlador porque obligaría a cada pantalla a conocer particularidades de infraestructura.

En la SPA, el cliente HTTP interpreta el sobre, conserva detalles de validación y convierte el error en un modelo común. Los formularios sitúan los problemas de campo junto al control y

muestran el resumen con rol accesible a `!ert`. Un conflicto operativo —por ejemplo, recurso ocupado— no se confunde con un problema de red; y un aviso que no impide la operación, como la capacidad, usa otro canal visual.

6.3 Sesión en el frontend

El Sprint 1 conservó token, identidad y caducidad en `localStorage` y adjuntaba `Authorization: Bearer` desde el cliente HTTP. Esa elección hizo verificable el [esquema andante](#) sin servidor de sesiones ni flujo de renovación, pero aumentaba el impacto de una vulnerabilidad [secuencias de comandos en sitios cruzados \(XSS\)](#): cualquier JavaScript ejecutado en el origen podía leer el token. Se documentó como decisión provisional y se concentró deliberadamente el acceso al almacenamiento en el contexto de autenticación y el cliente, para que el cambio posterior afectase a dos ficheros.

El endurecimiento sustituyó ese mecanismo por una cookie `HttpOnly` con `SameSite=Strict`. El atributo `Secure` se condiciona al entorno, no al esquema de cada petición: tras un proxy que termina TLS la API recibe la conexión interna como HTTP, así que decidirlo por esquema lo desactivaría precisamente en el único camino real. Un inicio de sesión sin HTTPS falla cerrado en lugar de crear una sesión insegura.

El intercambio no es gratuito y conviene enunciarlo: se reduce la exposición del token frente a XSS a cambio de introducir superficie [falsificación de petición en sitios cruzados \(CSRF\)](#). La operación en mismo origen y `SameSite=Strict` la cubren, porque una navegación o un formulario de terceros no adjuntan la cookie. La consecuencia práctica es que la SPA ya no puede leer su propia sesión, de modo que la reconstruye al cargar contra `GET /api/v1/auth/me` y la descarta mediante `POST /api/v1/auth/logout`. La ruta protegida muestra un estado de comprobación mientras esa consulta está en curso; sin él, recargar expulsaría al usuario antes de conocer si tiene sesión.

6.4 Catálogos: baja lógica y unicidad

Vehículos, conductores y clientes implementan RF-05, RF-06 y RF-07 con un patrón común: listado de activos, detalle, alta, edición y desactivación. RN-15 y RNF-03 impiden que una baja destruya información que ya participa en operaciones. Por eso `DELETE` expresa una acción de negocio reversible en los datos —cambiar `ISActive`— y no un borrado físico. Las consultas ordinarias ocultan inactivos, mientras que las proyecciones históricas pueden seguir resolviendo sus etiquetas.

No todos los campos merecen la misma regla de identidad. La matrícula y el código interno identifican funcionalmente un vehículo; el número de carné identifica a un conductor. El

servicio normaliza matrícula, códigos y carné a mayúsculas, recorta textos y convierte cadenas opcionales vacías en nulo antes de comprobar conflictos. La validación temprana produce mensajes de dominio y la base de datos refuerza el resultado con [índice único filtrados](#) sobre filas activas. La combinación evita tanto carreras triviales como diferencias de representación.

La unicidad solo entre activos permite reutilizar un identificador tras una baja, decisión acordada para los catálogos. No se aplicó a nombre de cliente ni código de empleado porque la entrevista no los declaró únicos. Tampoco se aplica a credenciales de usuario, cuya identidad histórica requiere unicidad global. Estas diferencias evitan imponer reglas inventadas por comodidad técnica.

Las relaciones desde `Shipment` usan `RESTRICT`: desactivar no afecta a la clave y un eventual borrado físico no podría dejar un envío sin su referencia original. Para crear o cambiar una asociación, en cambio, el servicio exige que el cliente o recurso esté activo. Esta asimetría satisface a la vez RN-03 —solo activos en operaciones nuevas— y RN-15 —historial conservado—.

La interfaz reutiliza estructura de lista, detalle y formulario, pero no una abstracción genérica que ocultase diferencias de campos y mensajes. Se prefirió repetición pequeña y explícita: tres módulos fáciles de seguir y adaptar frente a un metacomponente de formularios prematuro. Los flujos comparten el cliente, el contrato de error y las convenciones visuales.

6.5 Envíos: identificación, fechas y consultas

`Shipment` concentra RF-08 y RF-12. Su referencia se recorta, se transforma a mayúsculas y permanece única de forma global, incluso cuando el envío ha terminado, porque es el identificador utilizado en comunicación y trazabilidad. A diferencia de los catálogos, el envío no tiene baja lógica ni endpoint de borrado: su situación se expresa por estado y su historial debe permanecer consultable.

Origen, destino y recogida prevista son obligatorios. Cliente, entrega prevista, carga estimada y notas son opcionales para permitir planificar con información incompleta. RN-06 impide que la entrega prevista preceda a la recogida; se comprueba en la petición, en el servicio tras normalizar y mediante una restricción [SQL](#). La defensa en profundidad proporciona un error útil en el borde y protege la invariante frente a escrituras futuras que no recorran ese borde.

Las fechas de negocio se modelan como instantes [UTC](#) y PostgreSQL las almacena como `timestamp with time zone`. El normalizador acepta valores con sufijo `Z`, con desplazamiento explícito o sin zona. En el último caso el contrato interpreta UTC en vez de la zona local del servidor, eliminando resultados distintos entre equipos. La SPA convierte explícitamente entre los controles `datetime-local` y UTC; mostrar hora local es una

responsabilidad de presentación y no altera el instante persistido.

La consulta admite filtros combinables por estado, intervalo de recogida, cliente, vehículo y conductor. Los filtros se aplican antes del recuento y la paginación. El orden por recogida prevista y, como desempate, referencia produce páginas estables: sin un segundo criterio dos filas simultáneas podrían cambiar de página entre consultas equivalentes. Si la página pedida supera el total se devuelve vacía con los metadatos solicitados, conducta predecible para la SPA.

Los filtros del listado viven en la URL. Esto permite recargar, usar navegación atrás y compartir una vista operativa sin crear estado paralelo. Al editar, el backend devuelve la proyección del cliente incluso si ya está inactivo y la interfaz lo conserva como opción actual; no permite seleccionarlo para un envío nuevo, pero tampoco fuerza a borrar una relación histórica al modificar otro campo.

6.6 Operación: asignación y ciclo de vida

La operación de RF-09 y RF-10 se diseñó mediante acciones explícitas: PUT /shipments/{id}/assignment, DELETE sobre esa asignación y POST /shipments/{id}/status. Se descartó permitir que el [crear](#), [consultar](#), [actualizar](#) y [eliminar](#) (CRUD) general editase VehicleId, DriverId o Status: una actualización genérica facilitaría saltarse precondiciones y no expresaría la intención en el contrato.

La asignación es conjunta y solo se modifica mientras el envío está planificado. Vehículo y conductor forman una única decisión operativa; si falta uno, toda la solicitud falla y no existe un estado parcial creado por el servidor. Antes de escribir se verifica RN-03 y se consulta si cualquiera de los recursos participa en otro envío *Planned* o *InProgress*, excluyendo el actual. Así se aplica RN-04 y se permite sustituir un solo recurso enviando de nuevo la pareja completa.

Hasta el Sprint 6 la regla de [doble reserva](#) vivía solo en el servicio, lo que dejaba una ventana entre consulta y escritura: dos solicitudes simultáneas podían observar disponibilidad y confirmar ambas. El endurecimiento la cerró sin renunciar al mensaje útil, añadiendo la garantía por debajo en lugar de sustituir la comprobación. PostgreSQL sostiene ahora dos [índice único filtrados](#) únicos sobre Shipments, uno por vehículo y otro por conductor, limitados a los estados abiertos y a valores no nulos; los envíos sin asignar no colisionan porque el motor trata los nulos como distintos.

El reparto de responsabilidades es deliberado: la comprobación previa identifica qué envío ocupa el recurso y produce un conflicto descriptivo, mientras que el índice garantiza la corrección cuando dos peticiones la superan a la vez. La violación de unicidad se traduce al

mismo contrato 409 y al mismo código de error, de modo que la interfaz no distingue el origen. Al ser una garantía del motor y no del proveedor en memoria que usan las pruebas rápidas, se verifica con una clase dedicada que arranca PostgreSQL real en un contenedor y lanza dos asignaciones concurrentes.

RN-05 tiene semántica de aviso, no de prohibición. Si carga estimada y capacidad están informadas y la segunda es menor, la asignación se guarda y la respuesta contiene `capacityWarning`. No se persiste un booleano derivado, que quedaría obsoleto al cambiar carga o vehículo, ni se usa un diálogo de confirmación de dos pasos que duplicaría la operación. La advertencia es transitoria y visualmente distinta de un conflicto.

La *máquina de estados* de la Figura 6.1 constituye una lista cerrada. Desde *Planned* se puede iniciar o cancelar; iniciar exige ambos recursos. Desde *InProgress* se puede entregar o cancelar. *Delivered* y *Cancelled* son terminales, conforme a RN-07 y RN-08. Al iniciar se sella `ActualPickupAt`; al entregar, `ActualDeliveryAt`; ambos valores proceden del reloj UTC del servidor, no del cliente, para que el registro efectivo no dependa del equipo del operador.

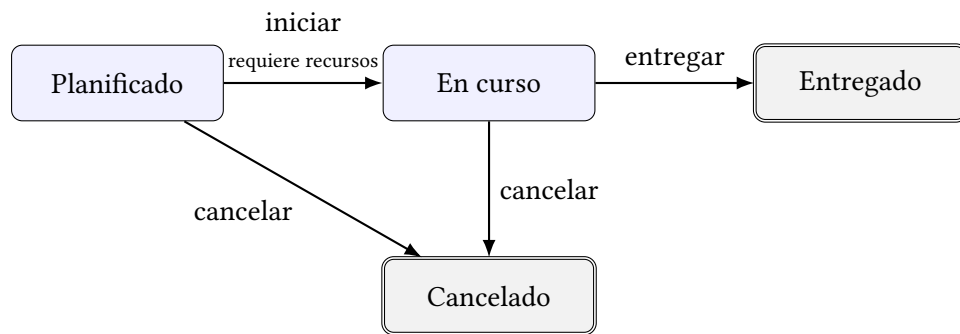


Figura 6.1: Máquina de estados cerrada del envío

El Flujo 3 de requisitos reúne estas decisiones en un recorrido completo. La Figura 6.2 muestra la secuencia lógica; cada mutación se valida en API y se confirma junto con su traza. No pretende detallar todas las respuestas HTTP, sino hacer visible qué componente asume cada responsabilidad.

6.7 Trazabilidad: historial inmutable

RF-11 modela la traza como subrecurso `/shipments/{id}/events`. La API solo permite alta y consulta: no hay edición ni borrado. Corregir silenciosamente un hecho destruiría el valor probatorio del historial; una rectificación futura debería registrarse como un nuevo evento, no reescribir el anterior.

`OccurredAt` representa cuándo sucedió el hecho de negocio y `CreatedAt` cuándo

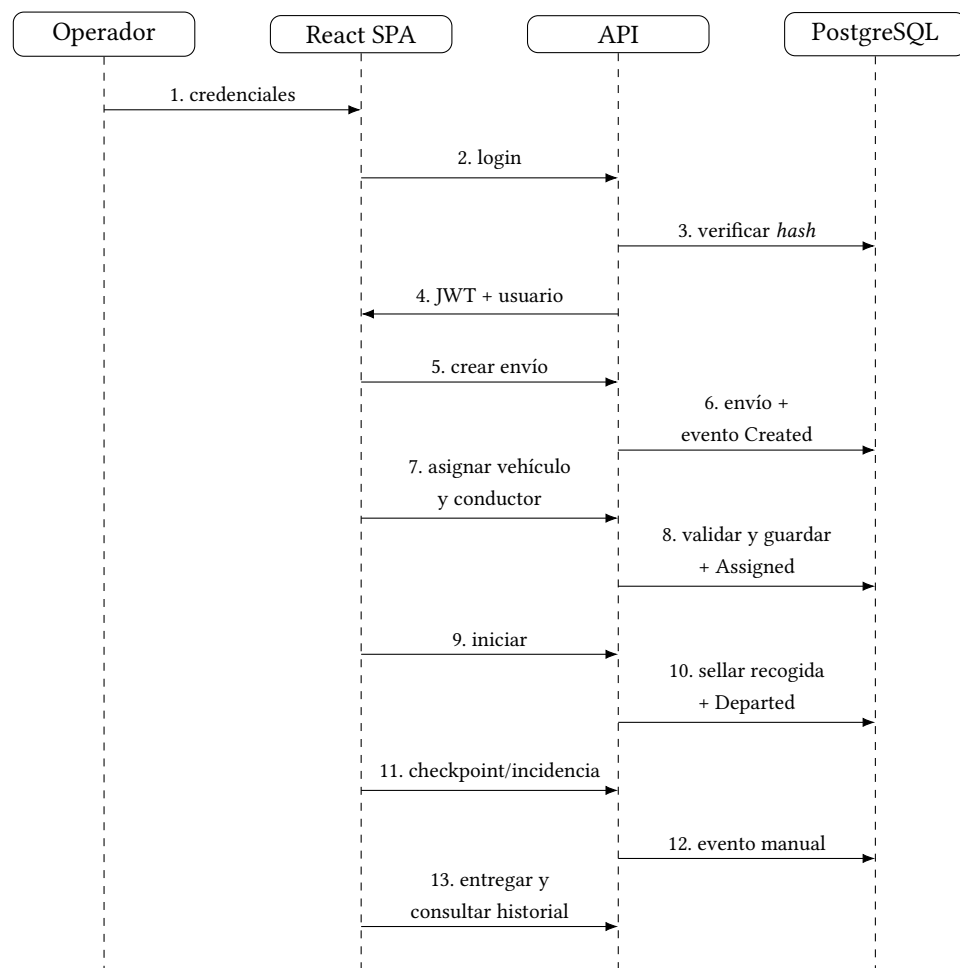


Figura 6.2: Secuencia resumida del Flujo 3: ejecución integral de un envío

se anotó. La separación permite registrar tarde una incidencia sin falsear su momento. Las consultas ordenan primero por ocurrencia y después por creación, criterio estable que produce una cronología coherente incluso con eventos retrospectivos. Las fechas manuales futuras se rechazan con una tolerancia pequeña para diferencias de reloj.

Los tipos *Created*, *Assigned*, *Unassigned*, *Departed*, *Delivered* y *Cancelled* están reservados al sistema. La petición pública solo acepta *Checkpoint* e *Incident*; así un cliente no puede fabricar una transición histórica sin cambiar el estado real. Las operaciones de envío añaden su evento automático al mismo contexto antes de un único `SaveChangesAsync`. La transacción implícita confirma ambos cambios o ninguno, preservando consistencia sin introducir un bus de eventos, interceptores o infraestructura desproporcionada.

RN-09 exige autoría. `ICurrentUser` encapsula la lectura del claim sub: los servicios dependen de una identidad mínima y las pruebas sustituyen el contexto web con facilidad. Se prefirió esta abstracción frente a leer `HttpContext` dentro del dominio o propagar un identificador por todas las firmas. `RecordedByUserId` admite nulo para trazas técnicas sin actor, aunque las operaciones autenticadas actuales registran al usuario.

La relación del evento con el usuario es restrictiva y conserva el autor aunque se desactive. La relación con el envío usa cascada como única excepción al criterio general, porque el evento forma parte del agregado y no tiene vida independiente. Dado que no existe borrado de envíos, la cascada funciona hoy como protección estructural y no como comportamiento accesible.

6.8 Administración e indicadores

RF-04 se publica tras la política real *Admin*. El servicio permite listar activos o incluir inactivos, consultar por identificador, crear, cambiar rol, cambiar activación y asignar una contraseña nueva a otra persona. Los inactivos permanecen direccionables para hacer posible la reactivación. No existe borrado ni reutilización de nombre o correo: a diferencia de matrícula o carné, las credenciales identifican a quien pudo originar eventos y su unicidad es global.

RN-12 impide dejar la aplicación sin administración. Antes de desactivar a un administrador activo o degradarlo, una guarda busca otro administrador activo. La misma regla cubre ambos caminos y una operación sin cambio conserva la *idempotencia*: devuelve el estado actual sin mutar. Existía aquí una carrera análoga a la de RN-04, porque dos degradaciones simultáneas podían observar cada una al otro administrador y confirmarse ambas.

A diferencia de RN-04, esta invariante no se expresa como restricción declarativa: no habla de una fila, sino de cuántas quedan. El endurecimiento la resolvió serializando el tramo crítico con un cerrojo de aviso de PostgreSQL mantenido dentro de la transacción, de modo que la segunda operación vuelve a evaluar la regla con el resultado de la primera ya confirmado y

recibe el conflicto. El cerrojo se toma únicamente al degradar o desactivar a un administrador activo, no en toda mutación de usuarios, y se condiciona a que el proveedor sea relacional para no alterar las pruebas en memoria.

RF-03 y RN-14 permiten cambiar la contraseña propia. La abstracción `ICurrentUser` localiza la cuenta activa; después se verifica la contraseña actual y se exige que la nueva sea distinta antes de volver a calcular el *hash*. La respuesta no devuelve *hash* ni nueva credencial, y el cambio incrementa la versión de sesión, de modo que las sesiones abiertas con la contraseña anterior dejan de ser válidas de inmediato.

Redactar el manual de usuario descubrió un hueco entre la entrevista y los requisitos: la clienta esperaba que un administrador reasignase una contraseña olvidada, pero RF-03 lo declaró fuera de alcance y RF-04 nunca lo recogió. No existía forma de hacerlo, porque el autoservicio exige la contraseña anterior y recrear la cuenta no puede reutilizar el nombre, cuya unicidad abarca también a los inactivos. RN-17 cierra ese hueco: un administrador asigna una contraseña nueva a otra persona sin conocer la anterior, la operación funciona sobre cuentas inactivas e incrementa igualmente la versión de sesión para cerrar las suyas. La propia cuenta queda excluida de esa vía y se rige por RN-14, de modo que nadie elude la confirmación de su contraseña actual pasando por la pantalla de administración.

RF-14 resume dos clases de información que no deben confundirse. Los contadores de envíos por estado son globales y describen la situación actual. La actividad de vehículos y conductores se limita a un periodo por `PlannedPickupAt`; las incidencias usan `OccurredAt`. Sin parámetros, el intervalo termina en el instante actual y comienza treinta días antes; si solo se indica fin, se conservan treinta días hacia atrás. Un rango invertido produce 400.

El resumen se calcula mediante agregaciones y proyecciones, no mediante una tabla de métricas que pudiera desincronizarse. Las etiquetas de recursos se resuelven sin exigir que sigan activos, preservando el significado histórico. La SPA explica qué cifras son globales y cuáles dependen del periodo; los contadores de estado enlazan al listado con el filtro correspondiente. Se aceptó el coste de consulta porque el contexto esperado es una empresa con decenas de recursos. Medición e índices adicionales pertenecen al endurecimiento si las pruebas revelan una necesidad real.

Desarrollo iterativo

El Capítulo 6 describe el sistema tal como quedó; este describe cómo llegó a serlo. Cada sección corresponde a un sprint y conserva el orden temporal, incluidas las decisiones que después se revisaron y las limitaciones que cada incremento dejó anotadas por escrito antes de resolverlas. Esa doble lectura es intencionada: un capítulo de diseño sin crónica presenta el resultado como si hubiera sido evidente desde el principio, y buena parte de lo que este trabajo pretende demostrar ocurre precisamente en la diferencia entre lo que se planificó y lo que la implementación obligó a corregir. Las capturas que acompañan a cada sprint no son ilustraciones decorativas sino la evidencia de que el incremento quedó utilizable en su momento de cierre, y por eso se conservan con el estado que tenía la interfaz entonces y no con el actual.

7.1 Sprint 1: cimientos y esqueleto autenticado

El incremento comenzó con una solución .NET y dos proyectos independientes, más una SPA TypeScript. En backend se implementaron la entidad de usuario, su configuración EF Core y la migración `InitialCreate`; el servicio de autenticación; los controladores de salud y sesión; JWT, CORS, correlación y tratamiento uniforme de excepciones.

En frontend se construyeron el cliente HTTP, un contexto de sesión persistente, la pantalla de login, el componente de error, la ruta protegida y un *layout* adaptable. Un administrador ve la futura entrada de usuarios, mientras que un operador recibe solo la navegación común. El resultado atraviesa navegador, proxy, API y PostgreSQL: constituye un *esqueleto andante* real.

Docker Compose orquesta base de datos, API y web; la API aplica migraciones al arrancar. GitHub Actions compila y prueba ambas aplicaciones y ejecuta la migración contra PostgreSQL. Esta infraestructura se mantendrá en todos los incrementos siguientes.

La Figura 7.1 muestra el resultado del incremento: la pantalla de acceso servida por Nginx

desde la composición de contenedores, no por el servidor de desarrollo. La distinción importa más de lo que aparenta, porque significa que el primer sprint ya atraviesa la misma frontera que atravesará el sistema desplegado —navegador, proxy, API y base de datos— y que los problemas de integración entre esas piezas se descubrieron en la primera semana y no en la última.

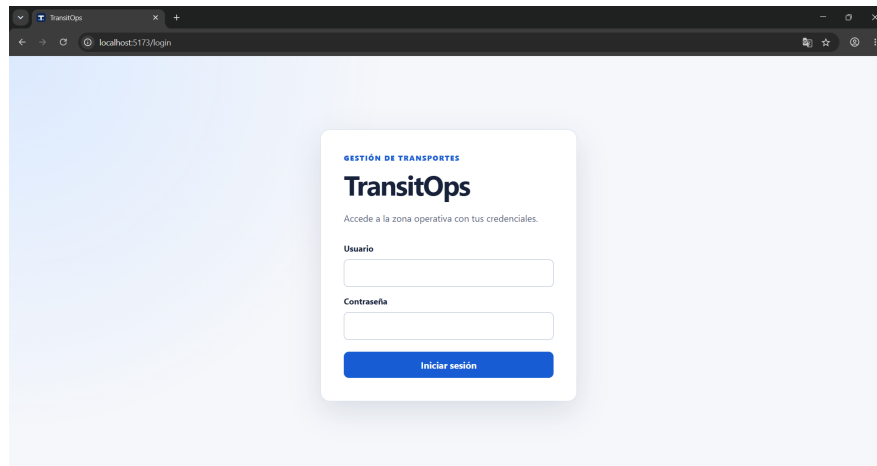


Figura 7.1: Pantalla de login servida por el *stack* de Docker Compose

7.2 Sprint 2: catálogos operativos

El segundo incremento incorporó vehículos, conductores y clientes como tres rebanadas coherentes. Cada catálogo dispone de listado, detalle, alta, edición y baja lógica tanto en la API como en la SPA. Los identificadores de negocio de vehículos y conductores se normalizan y son únicos entre registros activos, de modo que una baja conserva el historial y permite reutilizar el identificador.

La migración `AddCatalogTables` llevó el diseño a PostgreSQL sin alterar la autenticación existente. Los formularios reutilizan el contrato común de validación y muestran los conflictos junto al flujo que los origina. El cierre dejó 29 pruebas backend y 7 frontend correctas y verificó el flujo real a través de Nginx, API y PostgreSQL.

7.3 Sprint 3: envíos y visibilidad operativa

El tercer incremento añadió el agregado `Shipment`. Cada envío tiene una referencia única global, origen, destino y recogida prevista obligatorios; puede asociar cliente, entrega prevista, carga estimada y notas. Nace siempre planificado y no admite borrado: el ciclo de

estados se abordará en el incremento siguiente. Las claves foráneas restrictivas preservan los vínculos históricos con los catálogos.

La API normaliza referencias y fechas antes de persistir o filtrar. Los instantes se almacenan como UTC en `timestamp with time zone`; las entradas sin zona se interpretan según el contrato como UTC y las que incluyen desplazamiento conservan el instante. La regla que impide una entrega anterior a la recogida se comprueba en el contrato HTTP, el servicio y la base de datos.

La SPA ofrece alta, edición, detalle y una tabla paginada. Los filtros de estado, rango de recogida, vehículo y conductor viven en la URL. El formulario convierte de forma explícita entre `date-time-local` y UTC, muestra los errores por campo y conserva la asociación visible al editar un envío cuyo cliente ya fue dado de baja.

La automatización del cierre alcanzó 45 pruebas backend y 13 frontend, con compilaciones de producción y análisis estático correctos.

Las dos capturas del incremento documentan sus dos caras. En la Figura 7.2 aparece el listado con varios filtros aplicados simultáneamente; lo relevante no es la tabla sino la barra de direcciones, porque los criterios viajan en la *query string* y eso permite compartir una vista concreta o recuperarla al volver atrás sin que la aplicación tenga que recordar nada. La Figura 7.3 muestra un envío recién creado, todavía planificado y sin recursos asignados: es el estado en el que el tercer sprint deja necesariamente el agregado, porque la asignación y el ciclo de vida corresponden al siguiente. Que el detalle sea navegable antes de existir esas capacidades es lo que permitió validar el modelo de datos sin esperar a completarlo.

TransitOps

[Inicio](#) [Envíos](#) [Vehículos](#) [Conductores](#) [Clientes](#) [Usuarios \(próximamente\)](#)

admin · Administrador [Cerrar sesión](#)

OPERACIONES

Envíos

Nuevo envío

Estado

Planificado

Recogida desde

08/01/2026

Recogida hasta

08/31/2026

Vehículo

Todos

Conductor

Todos

Filtrar

Limpiar

REFERENCIA	ESTADO	ruta	RECOGIDA	ENTREGA	CLIENTE	ACCIONES
S3VERIFY-NAIVE-1785017733	Planificado	A → B	8/1/2026, 10:00:00 AM	—	—	Editar
S3VERIFY-OFFSET-1785017733	Planificado	A → B	8/1/2026, 10:00:00 AM	—	—	Editar
S3VERIFY-Z-1785017733	Planificado	A → B	8/1/2026, 10:00:00 AM	—	—	Editar
ENV-DEMO-S3	Planificado	A Coruña → Madrid	8/5/2026, 8:30:00 AM	8/5/2026, 6:00:00 PM	Sara	Editar

Página 1 de 1 · 4 envíos

Anterior

Siguiente

Figura 7.2: Listado de envíos con filtros de estado y fecha persistidos en la URL

TransitOps

[Inicio](#) [Envíos](#) [Vehículos](#) [Conductores](#) [Clientes](#) [Usuarios \(próximamente\)](#)

admin · Administrador [Cerrar sesión](#)

[← Volver al listado](#)

ENVÍO

ENV-DEMO-S3

Editar

ESTADO Planificado	ORIGEN A Coruña
DESTINO Madrid	RECOGIDA PREVISTA 8/5/2026, 8:30:00 AM
ENTREGA PREVISTA 8/5/2026, 6:00:00 PM	CLIENTE Sara
CARGA ESTIMADA 1250 kg	NOTAS Entrega prioritaria
VEHÍCULO Se asignará al operar el envío (próximamente)	CONDUCTOR Se asignará al operar el envío (próximamente)

Figura 7.3: Detalle de un envío planificado con cliente y carga estimada

7.4 Sprint 4: asignación y ciclo de vida

El cuarto incremento convirtió el detalle del envío en la pantalla de operación. La asignación trata vehículo y conductor como una unidad y solo se modifica mientras el envío está planificado. El servicio rechaza recursos inexistentes o dados de baja y consulta el resto de envíos planificados o en curso para impedir una doble reserva. La consulta excluye el propio envío, por lo que es posible cambiar únicamente el conductor sin provocar un falso conflicto con el vehículo que ya tenía asignado.

La capacidad insuficiente sigue una semántica distinta a la de los errores: la asignación se persiste y la respuesta incorpora un aviso transitorio. La SPA lo muestra en ámbar, separado del componente rojo de error. No se persiste una bandera derivada que podría quedar obsoleta al editar la carga o el vehículo.

La máquina de estados adopta una política cerrada: solo se permiten las cuatro transiciones dibujadas en la Figura 6.1. Entrar en curso requiere los dos recursos y sella la recogida real en UTC; entregar sella la entrega real. Entregado y cancelado son terminales, y ninguna operación devuelve el envío a planificado.

La migración `AddShipmentOperation` añadió las dos fechas reales y una restricción de coherencia. La API expone acciones explícitas para asignar, retirar la asignación y cambiar el estado; el CRUD general no puede alterar estos campos. En la interfaz, los botones se derivan del estado actual, las acciones irreversibles piden confirmación y un estado final muestra de forma explícita que ya no admite cambios.

Las cuatro capturas de este incremento recorren un envío completo y conviene leerlas en secuencia. La Figura 7.4 presenta el panel de asignación con el envío todavía planificado: vehículo y conductor aparecen en un único formulario porque la regla los trata como una pareja, y no como dos campos que se puedan guardar por separado. La Figura 7.5 recoge el caso que mejor distingue un aviso de un error: la carga supera la capacidad del vehículo, el mensaje aparece en ámbar y, pese a ello, la asignación se ha guardado; un operador que conoce su flota mejor que el sistema conserva la decisión. La Figura 7.6 muestra el mismo envío tras iniciarlo, ya con la hora real de recogida sellada y con el conjunto de acciones reducido a las que su estado admite. Y la Figura 7.7 cierra el recorrido: el envío entregado conserva ambas fechas reales y no ofrece ninguna acción de transición, porque la terminalidad no se comunica deshabilitando un botón sino retirándolo.

TransitOps Inicio **Envíos** Vehículos Conductores Clientes Usuarios (próximamente) admin - A

[← Volver al listado](#)

ENVÍO

S4-UI-001

ESTADO Planificado	ORIGEN A Coruña
DESTINO Madrid	RECOGIDA PREVISTA 5/8/2026, 10:00:00
ENTREGA PREVISTA —	RECOGIDA REAL —
ENTREGA REAL —	CLIENTE —
CARGA ESTIMADA 4500 kg	NOTAS Flujo integrado del Sprint 4
VEHÍCULO —	CONDUCTOR —

RECURSOS
Asignar vehículo y conductor
La asignación se confirma de forma conjunta y solo mientras el envío está planificado.

Vehículo
Selecciona un vehículo

Conductor
Selecciona un conductor

Asignar

Figura 7.4: Panel conjunto de asignación en un envío planificado

TransitOps Inicio **Envíos** Vehículos Conductores Clientes Usuarios (próximamente) admin - Administrador Cerrar sesión

[← Volver al listado](#)

ENVÍO

S4-UI-001 [Editar](#)

La capacidad del vehículo (3000 kg) es inferior a la carga estimada (4500 kg).

ESTADO Planificado	ORIGEN A Coruña
DESTINO Madrid	RECOGIDA PREVISTA 5/8/2026, 10:00:00
ENTREGA PREVISTA —	RECOGIDA REAL —
ENTREGA REAL —	CLIENTE —
CARGA ESTIMADA	NOTAS

Figura 7.5: Asignación guardada con aviso no bloqueante de capacidad

DESTINO Madrid	RECOGIDA PREVISTA 5/8/2026, 10:00:00
ENTREGA PREVISTA —	RECOGIDA REAL 30/7/2026, 23:48:11
ENTREGA REAL —	CLIENTE —
CARGA ESTIMADA 4500 kg	NOTAS Flujo integrado del Sprint 4
VEHÍCULO S4-UI-TRUCK	CONDUCTOR Conductor Sprint 4

CICLO DE VIDA
Actualizar estado

Marcar entregado Cancelar envío

Figura 7.6: Envío en curso con recogida real y acciones válidas

Entregado	A Coruña
DESTINO Madrid	RECOGIDA PREVISTA 5/8/2026, 10:00:00
ENTREGA PREVISTA —	RECOGIDA REAL 30/7/2026, 23:48:11
ENTREGA REAL 30/7/2026, 23:50:09	CLIENTE —
CARGA ESTIMADA 4500 kg	NOTAS Flujo integrado del Sprint 4
VEHÍCULO S4-UI-TRUCK	CONDUCTOR Conductor Sprint 4

CICLO DE VIDA
Actualizar estado

El envío está en un estado final y ya no puede cambiar.

Figura 7.7: Envío entregado, con fechas reales y sin acciones de transición

7.5 Sprint 5: trazabilidad de eventos

El quinto incremento completó el historial inmutable de cada envío. La entidad `ShipmentEvent` distingue la fecha en que sucedió un hecho (`OccurredAt`) de la fecha en que se registró (`CreatedAt`). Así, una incidencia puede anotarse más tarde sin falsear su hora real. El backend crea automáticamente los eventos de creación, asignación, retirada de asignación, salida, entrega y cancelación; el cliente HTTP solo puede solicitar puntos de control e incidencias, lo que impide insertar una entrega ficticia en la traza.

RN-09 introdujo por primera vez la identidad del actor en la capa de aplicación. El JWT conserva el identificador en el `claim sub`; en vez de repetir su lectura en cada controlador o acoplar los servicios a `HttpContext`, se definió la abstracción mínima `ICurrentUser`. Su implementación web usa `IHttpContextAccessor` y las pruebas la sustituyen por un doble trivial. La alternativa de pasar un `Guid` por todas las firmas era explícita, pero habría propagado una preocupación transversal por cada acción de envío; leer directamente el contexto dentro del servicio habría impedido aislarlo en pruebas.

Los eventos automáticos se añaden al mismo contexto antes de la única confirmación de EF Core. En consecuencia, la modificación del envío y su traza se guardan en la misma transacción implícita: no puede quedar una salida sin evento ni un evento de una transición rechazada. Se prefirieron llamadas explícitas a un *helper* privado frente a eventos de dominio, interceptores o un bus, porque el tamaño del dominio no justifica ocultar este efecto lateral.

La API representa el historial como subrecurso `/shipments/{id}/events`, sin edición, borrado ni paginación. La consulta ordena por fecha de ocurrencia y desempata por fecha de creación, proyectando también el nombre de usuario. La migración `AddShipmentEvents` añadió un índice compuesto para ese patrón de lectura y un índice por tipo que podrá reutilizar el resumen estadístico del Sprint 6.

La SPA integra una línea de tiempo en el detalle. Los eventos automáticos forman la espina dorsal y los manuales se diferencian visualmente; las incidencias reciben énfasis rojo. El formulario permite indicar una fecha pasada, transforma `datetime-local` a UTC y rechaza fechas futuras con dos minutos de tolerancia. Tras el alta inserta la respuesta en su posición cronológica sin recargar; tras una acción operativa vuelve a consultar el historial para incorporar el evento automático.

La Figura 7.8 muestra esa cronología ya poblada. La distinción visual entre los eventos que el sistema generó por sí mismo y los que una persona anotó es la propiedad central de la pantalla: permite responder si un hecho consta porque el sistema lo observó o porque alguien lo declaró, que es una pregunta distinta y a menudo la relevante al reconstruir una incidencia. La Figura 7.9 recoge el formulario de alta manual, donde se aprecia que la fecha del hecho es un campo editable y no la hora del registro; sin esa separación, anotar por la tarde una

incidencia de la mañana falsearía la cronología.

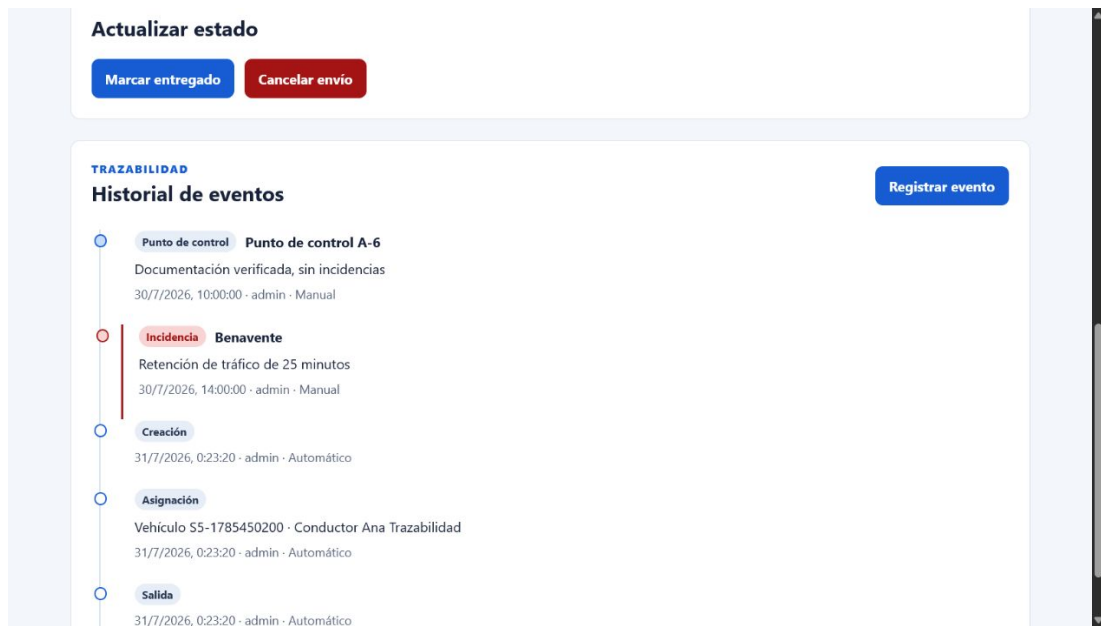


Figura 7.8: Línea de tiempo con eventos automáticos y manuales del Sprint 5

The screenshot shows a web form for manual event registration. The form is titled "Historial de eventos" and includes a "Cerrar formulario" button in the top right corner. The form fields are as follows:

- Tipo de evento:** A dropdown menu with "Punto de control" selected.
- Fecha y hora:** A date and time input field showing "31/07/2026 00:24" with a calendar icon.
- Ubicación (opcional):** A text input field.
- Notas (opcional):** A large text area for notes.

At the bottom of the form are two buttons: "Guardar evento" (Save event) and "Cancelar" (Cancel).

Below the form, there is a breadcrumb trail: "Punto de control" > "Punto de control A-6". Below this, it says "Documentación verificada, sin incidencias" and "30/7/2026, 10:00:00 - admin - Manual".

Figura 7.9: Formulario de registro de un evento manual

7.6 Sprint 6: administración e indicadores

El sexto incremento cerró los requisitos funcionales de prioridad media sin modificar el esquema. La entidad `AppUser`, diseñada y migrada en el Sprint 1, ya contenía rol, estado y marcas temporales; el resumen se obtiene mediante agregaciones sobre envíos y eventos. Esta ausencia de migración es un resultado del diseño anticipado del modelo, no una reducción de alcance.

La administración de usuarios se separó del servicio de autenticación y se situó tras la política `Admin`. Permite alta, cambio de rol, desactivación y reactivación, pero no borrado. Usuario y correo conservan unicidad global aunque la cuenta esté inactiva, porque identifican a la persona en el historial. RN-12 se aplica antes de cualquier mutación mediante una guarda común: desactivar al último administrador activo o degradarlo a operador devuelve un conflicto `last_admin_protected`. La SPA oculta el módulo a operadores y una ruta específica redirige también cualquier navegación directa; la API continúa siendo la autoridad y devuelve 403.

El cambio de contraseña propia reutiliza `ICurrentUser`: carga la cuenta activa, verifica el *hash* de la contraseña actual y rechaza tanto una credencial incorrecta como una nueva contraseña idéntica. El JWT no se sustituye ni se revocan sesiones previas. Del mismo modo, un token ya emitido sobrevive temporalmente a una desactivación o cambio de rol. Es una limitación consciente de la autenticación sin estado, acotada por la caducidad del token, que se revisará junto con el almacenamiento en `localStorage` durante el endurecimiento del Sprint 7.

El endpoint `/summary` combina tres consultas. Los contadores por estado son globales y representan la situación actual. La actividad de vehículo y conductor usa la fecha prevista de recogida dentro de un periodo configurable —treinta días por defecto—, mientras que las incidencias se cuentan por `OccurredAt`. Las etiquetas se resuelven sin filtrar recursos inactivos, de modo que una baja no borra actividad histórica. En el Inicio de la SPA, cada contador enlaza al listado de envíos ya filtrado y la interfaz explica expresamente qué cifras dependen del periodo.

La Figura 7.10 recoge ese panel. Su diseño responde a lo que la clienta pidió en la entrevista, que era ver la situación de un vistazo y no un informe elaborado, y por eso los indicadores son recuentos y no gráficas. La decisión con más consecuencias, sin embargo, no se ve en la captura: cada cifra es un enlace al listado ya filtrado por el criterio que la produjo. Eso convierte el resumen en un punto de entrada al trabajo operativo en lugar de un panel que hay que leer y luego traducir a mano a una búsqueda.

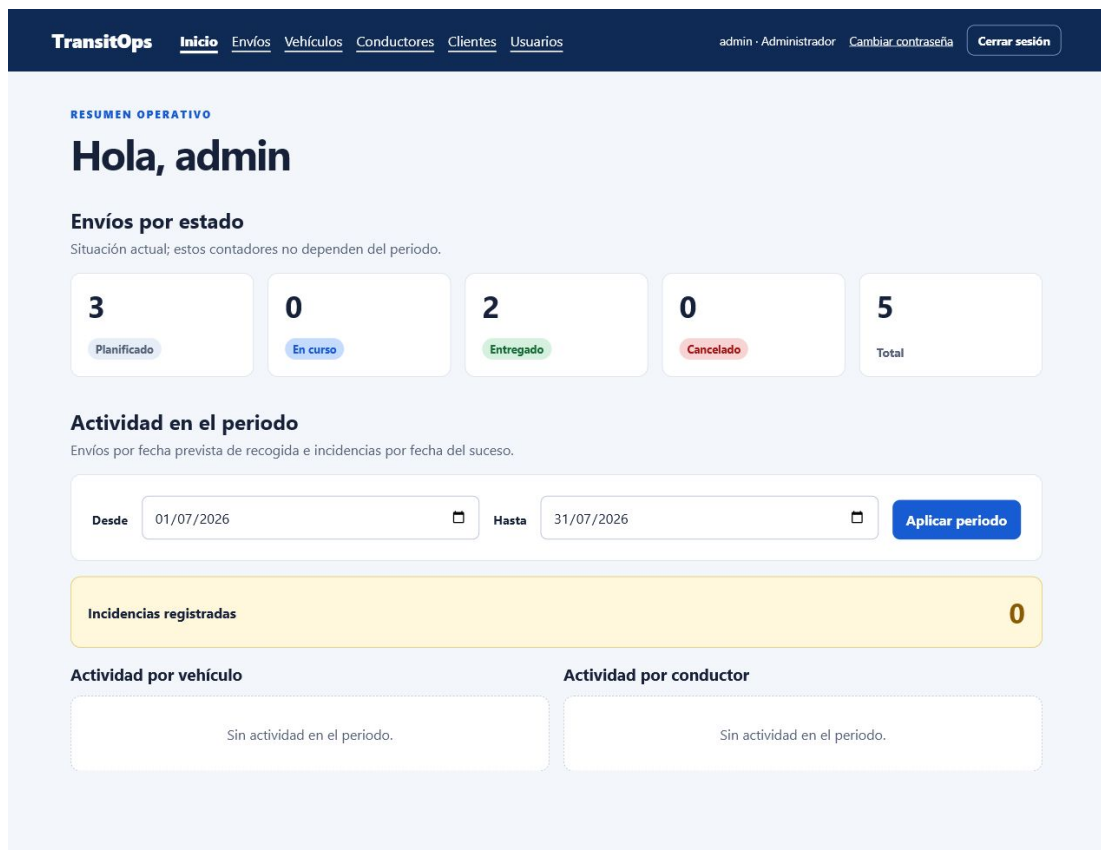


Figura 7.10: Panel operativo con situación actual y actividad del periodo

7.7 Sprint 7: endurecimiento, pruebas de sistema y despliegue

Con RF-01–RF-14 implementados, el séptimo incremento no añadió funcionalidad sino que atacó lo que los seis anteriores habían dejado anotado como deuda. Por tamaño se dividió en cuatro tramos consecutivos, cada uno con su propio cierre, y el orden entre ellos respondió a una decisión concreta: las pruebas de sistema se escribieron *antes* del cambio de sesión, para que actuaran como red de seguridad del refactor más invasivo. Al recorrer la interfaz, esas pruebas son indiferentes a si el token viaja en almacenamiento local o en una cookie; escritas después no habrían protegido nada.

Concurrencia sostenida por el motor

El primer tramo cerró las dos carreras declaradas en los Sprints 4 y 6. La regla anti-doble reserva pasó a estar garantizada por dos índices únicos parciales sobre los envíos, uno por vehículo y otro por conductor, limitados a los estados abiertos; la protección del último administrador se serializó con un cerrojo de aviso mantenido dentro de la transacción. La comprobación previa en el servicio se conservó en ambos casos, porque es la que produce un mensaje útil: el índice aporta corrección, el servicio aporta explicación.

Verificar esto obligó a ampliar la estrategia de pruebas. Ni un índice filtrado ni un cerrojo existen en el proveedor en memoria que usan las pruebas rápidas, así que se incorporó una clase que arranca PostgreSQL real en un contenedor efímero y lanza dos operaciones simultáneas. Es el primer punto del proyecto donde una regla no podía demostrarse sin pagar el coste de infraestructura de prueba.

Pruebas de sistema y dependencias

El segundo tramo automatizó los cuatro flujos de negocio con Playwright sobre la composición de contenedores. Las pruebas son independientes entre sí y no reinician la base: generan datos únicos por ejecución, algo obligatorio y no cosmético, porque las credenciales de usuario y las referencias de envío son únicas de forma global e incluyen a los registros inactivos. El arranque del primer administrador, que solo puede tener éxito una vez por base de datos, se resolvió en la preparación global tolerando el conflicto.

La misma revisión abordó las dependencias del frontend. La auditoría encontró cinco avisos, uno de ellos en una dependencia de producción —una vulnerabilidad de falsificación de petición en el enrutador— y el resto en el árbol de desarrollo y pruebas. Se actualizaron por versiones compatibles en lugar de aplicar correcciones automáticas mayores, y se documentó el alcance de cada cadena para distinguir lo que llega a la imagen servida de lo que no.

Sesión endurecida

El tercer tramo pagó la deuda más visible del Sprint 1. El token dejó de ser puramente autocontenido: incorpora una versión de sesión que se comprueba en cada petición protegida, de modo que cambiar la contraseña, desactivar la cuenta o modificar el rol invalidan de inmediato los tokens anteriores. Y el almacenamiento local dio paso a una cookie `HttpOnly` con `SameSite=Strict`, barata de adoptar porque la aplicación siempre ha operado en mismo origen.

El cambio tuvo dos consecuencias que no eran evidentes al planificarlo. La respuesta de acceso dejó de contener el token, lo que alteró el contrato y las pruebas que lo simulaban; y la interfaz perdió la capacidad de leer su propia sesión, de modo que necesitó un punto final nuevo para reconstruirla al recargar. Sin él, refrescar la página expulsaba al usuario. Ambas se descubrieron al implementar, no al diseñar, y quedan registradas porque ilustran el tipo de coste que un cambio de mecanismo arrastra más allá de su propia superficie.

Despliegue accesible

El cuarto tramo llevó la aplicación a una máquina virtual Ubuntu con Docker Compose, accesible por una dirección HTTPS temporal mediante un túnel saliente, sin publicar ningún puerto ni abrir el router. La entrega quedó automatizada hasta el registro de contenedores y la máquina consume desde allí, con un temporizador que comprueba novedades cada cinco minutos. El Capítulo 9 detalla la topología y el procedimiento.

Cerrado ese tramo, una revisión completa del incremento localizó trece defectos de coherencia, ninguno funcional: en su mayoría código y configuración que el cambio de sesión había dejado sin uso pero aparentando seguir activos —un punto final duplicado, una política de intercambio de recursos entre orígenes inservible, un ayudante de pruebas que leía un campo eliminado— además de una vulnerabilidad de severidad alta que había entrado de forma transitiva con la infraestructura de pruebas del primer tramo. Doce se corrigieron; el restante, la no identidad entre las imágenes probadas y las publicadas, se documentó como limitación conocida. Que una revisión posterior encuentre este tipo de residuo es esperable cuando un sprint sustituye un mecanismo transversal, y registrarlo es más útil que presentar el incremento como limpio desde el principio.

Sprint 8 — Cierre

El último incremento consolida esta memoria, produce el material de defensa y revisa que la documentación del repositorio describa el sistema realmente entregado.

Capítulo 8

Validación

Este capítulo responde a una pregunta concreta: por qué debe creerse que el sistema cumple lo que los capítulos anteriores afirman. Primero expone la estrategia de pruebas y el criterio con el que se eligió cada nivel; después recorre cómo creció la cobertura incremento a incremento, con el detalle de qué comportamiento quedó cubierto en cada uno; y termina con la validación del sistema ensamblado y la del entorno desplegado, que son las únicas capaces de demostrar propiedades que ninguna prueba aislada alcanza. Se declaran también, de forma explícita, las comprobaciones que quedaron fuera de la automatización, porque una relación de pruebas que omite sus huecos informa peor que una más corta que los enuncia.

8.1 Estrategia de pruebas

La estrategia combina cuatro niveles, cada uno elegido por lo que puede demostrar y por lo que cuesta ejecutar. Las pruebas de integración de API arrancan la aplicación completa con `WebApplicationFactory` y sustituyen solo PostgreSQL por una base en memoria aislada, lo que las hace rápidas y aptas para la mayoría de reglas de negocio. Las pruebas de componentes y rutas de frontend simulan la API y manipulan la interfaz como lo haría una persona usuaria.

El endurecimiento añadió dos niveles más, porque los anteriores no podían demostrar lo que introdujo. Las garantías que dependen del motor relacional —índices únicos parciales y cerrojos— no las respeta un proveedor en memoria, así que se verifican con una clase que arranca PostgreSQL real en un contenedor efímero mediante `Testcontainers`. Y los cuatro flujos de negocio, que atraviesan sesión, navegación y datos entre pantallas, se ejercitan con `Playwright` contra la composición completa de contenedores. La proporción es deliberada: la inmensa mayoría de las pruebas siguen siendo rápidas y aisladas, y solo lo que no se puede demostrar de otro modo paga el coste de un contenedor.

8.2 Evolución de la cobertura por incremento

Las tablas de este apartado recogen, sprint a sprint, qué comportamiento concreto quedó cubierto y en qué capa. Están redactadas en términos de comportamiento observable y no de clases o ficheros, porque el propósito es que puedan contrastarse con los requisitos del Capítulo 4 sin conocer la estructura interna del código.

La Tabla 8.1 corresponde al primer incremento. Su contenido revela una decisión de la que dependen los seis sprints siguientes: la autenticación y la autorización se probaron desde el principio, incluidos los rechazos 401 y 403, en lugar de añadirse como una capa posterior. Un esqueleto que se autentica de verdad obliga a que toda capacidad nueva nazca ya dentro de un contexto con permisos.

Tabla 8.1: Pruebas automatizadas del Sprint 1

Comportamiento	Capa	Resultado
Primer administrador y <i>hash</i> de contraseña	Backend	Correcto
Bloqueo de un segundo bootstrap	Backend	Correcto
Login válido, erróneo y usuario inactivo	Backend	Correcto
Rutas protegidas 401 y 403	Backend	Correcto
Contrato de validación	Backend	Correcto
Redirección sin sesión	Frontend	Correcto
Login, persistencia y error visible	Frontend	Correcto
Navegación adaptada al rol administrador	Frontend	Correcto

La ejecución local al cierre del incremento produjo dieciséis pruebas backend, organizadas por servicio y controlador, y cuatro frontend correctas, además de la compilación de producción de ambos proyectos. La CI repite las mismas comprobaciones y valida la migración sobre PostgreSQL 16.

La Tabla 8.2 agrupa los incrementos segundo y tercero, porque comparten un mismo tipo de riesgo: la conservación de datos históricos y la corrección de las representaciones. Dos filas concentran el valor del conjunto. La de unicidad y reutilización tras baja demuestra que un identificador queda libre cuando su registro se da de baja pero el historial que lo menciona no se pierde, que es la tensión que resuelve la baja lógica. Y la de normalización UTC comprueba los tres formatos de entrada posibles, porque una fecha que cambia de significado al cruzar de navegador a base de datos produce errores que no se manifiestan hasta que alguien opera en otro horario.

Tabla 8.2: Validación acumulada de los Sprints 2 y 3

Comportamiento	Capa	Resultado
CRUD y baja lógica de los tres catálogos	Backend/ Frontend	Correcto
Unicidad y reutilización tras baja	Backend	Correcto
Alta, consulta y edición de envíos	Backend/ Frontend	Correcto
Normalización UTC de formatos Z, sin zona y con offset	Backend	Correcto
Regla de fechas y errores por campo	Backend/ Frontend	Correcto
Filtros combinables y paginación estable	Backend/ Frontend	Correcto
Cliente activo y conservación del vínculo histórico	Backend	Correcto

Tras el Sprint 2 se ejecutaban 29 pruebas backend y 7 frontend. El Sprint 3 añadió 16 y 6 respectivamente, elevando el total a 45 pruebas backend y 13 frontend, todas correctas junto con el *lint* y las compilaciones de producción.

La Tabla 8.3 cubre el incremento con más reglas de negocio del proyecto. Conviene fijarse en la columna de capa: es la primera vez que aparecen las marcas de sistema y de PostgreSQL, y no por afán de exhaustividad. La regla anti-doble reserva y el sellado de fechas reales dependen del motor relacional, así que comprobarlas únicamente contra un proveedor en memoria habría producido una confianza infundada.

El cierre del Sprint 4 elevó el total a 74 pruebas backend y 19 frontend. Además de las suites, se ejecutó el flujo integrado mediante Nginx, API y PostgreSQL 16: se verificaron la migración, las proyecciones SQL, el aviso de capacidad, el conflicto que identifica el envío ocupante y el bloqueo visual de un estado final. La prueba en navegador descubrió además un

Tabla 8.3: Validación del Sprint 4

Comportamiento	Capa	Resultado
Asignación conjunta y retirada idempotente	Backend/ Frontend	Correcto
Recursos activos y exclusión del propio envío	Backend	Correcto
Anti-doble reserva en estados abiertos	Backend/ Sistema	Correcto
Aviso no bloqueante de capacidad	Backend/ Frontend	Correcto
Transiciones válidas, inválidas y terminales	Backend/ Frontend	Correcto
Sellado UTC de recogida y entrega reales	Backend/ PostgreSQL	Correcto
Proyección de matrícula y conductor en PostgreSQL	Integración	Correcto

caso de fecha inválida que podía dejar el formulario pendiente; se corrigió y quedó cubierto por una prueba de regresión.

La Tabla 8.4 corresponde al historial de eventos, cuyo requisito de inmutabilidad exige probar tanto lo que el sistema permite como lo que debe impedir. Las dos filas que expresan eso son la de tipos del sistema bloqueados para el cliente —que evita insertar una entrega ficticia en la traza— y la de eventos automáticos en la misma operación, que comprueba que una transición y su registro se guardan juntos o no se guardan.

El Sprint 5 elevó el total a 96 pruebas backend y 24 frontend. Las pruebas de controlador verifican la asociación del evento con el usuario real del token, no solo con un doble; las de servicio comprueban el orden estable, el aislamiento entre envíos y que una transición rechazada no deja rastro. El *lint* y las compilaciones de producción permanecieron correctos.

La Tabla 8.5 cierra la cobertura funcional. Su fila más significativa es la protección del último administrador, porque describe una regla cuyo incumplimiento dejaría el sistema inutilizable de forma irreversible: sin ningún administrador activo no existe vía para crear otro, ya que el arranque solo funciona una vez. Las dos filas de agregaciones sobre PostgreSQL responden a que los contadores del resumen se calculan en la base de datos y no en memoria.

El Sprint 6 elevó el total a 127 pruebas backend y 30 frontend. Además de las suites, se construyeron ambos contenedores y se comprobaron las agregaciones sobre PostgreSQL 16 con periodo omitido, explícito y fechas sin zona. El flujo de navegador verificó el alta de un

Tabla 8.4: Validación del Sprint 5

Comportamiento	Capa	Resultado
Alta y consulta inmutable por envío	Backend/ Frontend	Correcto
Actor obtenido del claim sub	Backend/ HTTP	Correcto
Normalización UTC y tolerancia de futuro	Backend/ Frontend	Correcto
Orden total por ocurrencia y creación	Backend/ PostgreSQL	Correcto
Eventos automáticos en la misma operación	Backend/ Integración	Correcto
Tipos del sistema bloqueados para el cliente	Backend/ HTTP	Correcto
Inserción retroactiva sin recargar historial	Frontend	Correcto
Refresco tras asignación o transición	Frontend	Correcto

operador, la ausencia del enlace administrativo, la redirección de `/usuarios`, el formulario de contraseña y el mensaje de conflicto al intentar desactivar al único administrador. No se generó ninguna migración y EF confirmó que la base estaba actualizada.

8.3 Validación del sistema completo

El endurecimiento elevó el total a 139 pruebas backend, 33 frontend y los cuatro flujos de sistema. El incremento no es solo de cantidad: las pruebas nuevas de backend comprueban la invalidación de sesión tras cambiar la contraseña, desactivar la cuenta y cambiar el rol, y una clase específica lanza dos asignaciones concurrentes contra PostgreSQL real para verificar que el índice único parcial rechaza la segunda con el conflicto esperado, además de dos degradaciones simultáneas del último administrador. Esa comprobación es imposible con el proveedor en memoria, que no aplica ni índices filtrados ni cerrojos de aviso.

La suite de frontend se reorganizó en cinco ficheros por área —sesión, catálogos, resumen, usuarios y envíos— espejando la separación que el backend ya tenía entre controladores y servicios. Además del beneficio de lectura, la ejecución pasó de unos veintidós segundos a menos de tres, porque el ejecutor paraleliza entre ficheros y antes existía uno solo.

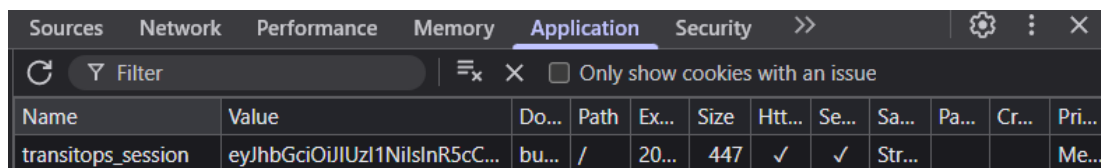
Las pruebas de sistema del Sprint 7 recorren los cuatro casos de uso con Playwright sobre

Tabla 8.5: Validación del Sprint 6

Comportamiento	Capa	Resultado
Alta, listado y reactivación de usuarios	Backend/ Frontend	Correcto
Política administrativa y ruta directa de operador	HTTP/ Frontend	Correcto
Protección del último administrador	Backend/ HTTP	Correcto
Cambio efectivo de contraseña propia	Backend/ HTTP/ Frontend	Correcto
Contadores completos, incluidos estados a cero	Backend	Correcto
Actividad e incidencias con límites inclusivos	Backend/ PostgreSQL	Correcto
Fechas sin zona y periodo por defecto	PostgreSQL	Correcto
Enlaces del panel hacia filtros operativos	Frontend	Correcto

la composición de contenedores. Sobre la sesión comprueban que la cookie no es legible desde `document.cookie`, evidencia observable del atributo `HttpOnly`, y que sobrevive a una recarga completa de la página. La colección ejecutable de la API añade la verificación directa de la cabecera `Set-Cookie`, incluidos `HttpOnly` y `SameSite=Strict`, y la reautenticación posterior al cambio de contraseña, cuya invalidación por versión de token cubren además las pruebas de servicio del backend.

Queda fuera de esa automatización el atributo `Secure`. La integración continua no dispone de terminación TLS y ejecuta la API con el entorno `Development`, en el que la aplicación emite deliberadamente la cookie sin esa marca para que el recorrido funcione por HTTP; `Secure` solo se activa fuera de `Development`. Su verificación es, por tanto, manual, y se realizó sobre el despliegue accesible con HTTPS. La Figura 8.1 recoge esa inspección: las herramientas del navegador muestran la cookie `transitops_session` con las tres marcas `HttpOnly`, `Secure` y `SameSite=Strict` sobre la dirección pública del túnel. Se documenta de forma explícita porque implica que una de las propiedades endurecidas en el séptimo incremento se ejercita automáticamente en el único entorno donde permanece desactivada, y su comprobación depende de una inspección puntual y no de la suite.



Name	Value	Do...	Path	Ex...	Size	Htt...	Se...	Sa...	Pa...	Cr...	Pri...
transitops_session	eyJhbGciOiJIUzI1NiIsInR5cC...	bu...	/	20...	447	✓	✓	Str...			Me...

Figura 8.1: Cookie de sesión sobre el despliegue con HTTPS: `HttpOnly`, `Secure` y `SameSite=Strict`

Todas las suites descritas se ejecutan además en la integración continua, que las repite sobre una máquina limpia y sin acceso al entorno de desarrollo. La Figura 8.2 muestra una de esas ejecuciones completa. Su valor no es decorativo: mientras una ejecución local demuestra que las pruebas pasan en el equipo de quien las escribió, esta demuestra que el repositorio se reconstruye y se valida a partir de lo versionado, sin depender de ningún fichero, variable o herramienta presente solo en esa máquina.

8.4 Validación sobre el entorno desplegado

La validación sobre el entorno desplegado completó los criterios que no pueden observarse en local. La aplicación respondió por HTTPS con certificado válido; la comprobación de salud devolvió estado correcto a través del túnel; el punto final retirado en el saneado responde ya con ausencia y el de reconstrucción de sesión con falta de autenticación, lo que confirma que el artefacto desplegado corresponde al código revisado; y una petición con origen ajeno no

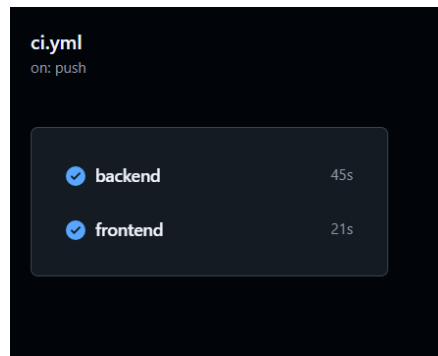


Figura 8.2: Ejecución de la integración continua en verde tras publicar la rama

obtiene cabeceras de intercambio entre orígenes, coherente con la eliminación de esa política. La revisión y los *digests* que el script imprime permiten atar cada una de esas observaciones a un *commit* concreto.

Esas cuatro comprobaciones comparten una propiedad que conviene enunciar: ninguna de ellas podría haberse sustituido por una prueba automatizada del repositorio, porque todas verifican el sistema tal como quedó configurado en su destino y no tal como se construye. Es la razón de que el capítulo no termine con la suite en verde, sino aquí.

Despliegue y reproducibilidad

Este capítulo describe cómo se ejecuta el sistema, tanto en la máquina de quien lo desarrolla como en un entorno accesible desde internet. Se ocupa de tres preguntas encadenadas: qué se necesita para levantar la aplicación, dónde se decidió alojarla y por qué, y cómo llega a esa máquina una versión nueva sin intervención manual. Las respuestas comparten un criterio: el entorno accesible no es una configuración paralela mantenida aparte, sino la misma composición de contenedores con otros parámetros, porque dos descripciones distintas del mismo sistema divergen en cuanto una de las dos deja de usarse. El capítulo termina enunciando los límites de lo conseguido, que son los que separan una demostración reproducible de un servicio en explotación.

9.1 Reproducibilidad local

Desde el primer sprint, `docker compose up --build` crea tres servicios. PostgreSQL mantiene un volumen persistente y expone una comprobación de salud; la API espera a que la base esté disponible, aplica migraciones y escucha en el puerto 8080; Nginx sirve la SPA en el puerto 5173 y reenvía `/api` al backend. El fichero `.env.example` enumera las variables que deben sustituirse y `.env` queda excluido del control de versiones, de modo que claves JWT, token de arranque y contraseña de base de datos nunca viajan en el repositorio.

Esa composición sigue siendo el entorno de desarrollo y el que ejecuta la integración continua. El despliegue accesible no la sustituye: la reutiliza con otro fichero y otros parámetros.

9.2 Destino y topología

El destino elegido es una máquina virtual Ubuntu Server 24.04 LTS con Docker Compose, alojada en el equipo del autor. Se descartó deliberadamente una plataforma gestionada con infraestructura como código: el objetivo de este trabajo es el ciclo de vida del software, y un

entorno de demostración reproducible desde un documento aporta más a esa tesis que una cuenta en la nube que un tribunal no puede recrear. La decisión tiene además una consecuencia deseable: el procedimiento que se documenta funciona igual sobre cualquier host con Docker, y cambiar de destino solo cambia la máquina.

La máquina se dimensionó deliberadamente al mínimo: un procesador virtual, 2 GB de memoria y 15 GB de disco. Los cuatro contenedores —PostgreSQL, API, servidor web y túnel— conviven en ese presupuesto, y un ciclo completo del despliegue periódico consume alrededor de un segundo de CPU sobre ese único núcleo. No constituye una medición de rendimiento bajo carga, que este trabajo no realizó, pero sí respalda de forma concreta la proporcionabilidad que pide RNF-06: la aplicación prevista para una empresa con decenas de vehículos y conductores no necesita infraestructura compleja para funcionar.

La composición de despliegue define cuatro servicios —base de datos, API, servidor web y túnel— y *no publica ningún puerto en la máquina anfitriona*, tampoco el de PostgreSQL. Los tres primeros se comunican por la red privada de Compose; el cuarto, `cloudflared`, establece una conexión saliente con Cloudflare y ofrece una [localizador uniforme de recursos \(URL\) protocolo de transferencia de hipertexto seguro \(HTTPS\)](#) temporal. No hay, por tanto, ninguna regla de entrada que abrir en el router ni superficie HTTP alternativa que quede a medio funcionar. El diagnóstico se realiza desde dentro de la máquina, consultando la salud a través del contenedor web.

La Figura 9.1 muestra la aplicación respondiendo en esa dirección pública. La captura documenta dos hechos a la vez: que el sistema es alcanzable desde fuera de la red donde se ejecuta, y que lo es por HTTPS con un certificado que el navegador acepta sin advertencias, pese a que en la máquina no se instaló ningún certificado ni se configuró ningún servidor TLS. Ambas propiedades proceden de la misma decisión de topología, y es la razón de que el túnel sustituya a la apertura de puertos en lugar de complementarla.

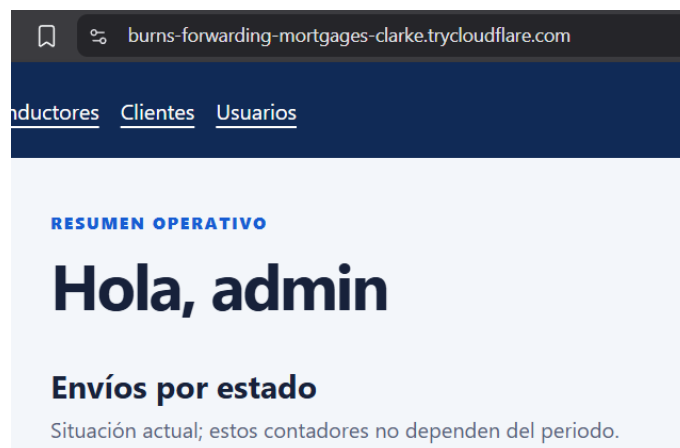


Figura 9.1: Aplicación accesible por la dirección pública del túnel

La terminación [seguridad de la capa de transporte \(TLS\)](#) ocurre en Cloudflare, de modo que el tramo interno entre el túnel y Nginx es HTTP. Esa es la razón por la que el atributo `Secure` de la cookie se condiciona al entorno de ejecución y no al esquema de la petición: decidirlo por esquema lo desactivaría precisamente en el único camino real de acceso. La API se ejecuta con el entorno `Production` y la cookie conserva la marca.

9.3 Entrega continua basada en *pull*

GitHub Actions no puede alcanzar la máquina virtual, que no tiene dirección pública ni puertos entrantes. En lugar de forzar esa conectividad, la entrega se invierte: el flujo de trabajo construye las dos imágenes y las publica en el registro de contenedores de GitHub solo después de superar compilación, pruebas unitarias, pruebas de integración y los cuatro flujos de sistema. La máquina virtual las descarga cuando le corresponde.

Esta inversión tiene una propiedad que conviene enunciar, porque no es un efecto colateral sino el motivo de la elección: *la existencia de la imagen es la prueba de que la validación pasó*. No hay forma de desplegar un artefacto que no haya superado la suite, porque el artefacto no llega a existir. Las imágenes se etiquetan con el identificador del *commit* además de `latest`, y llevan etiquetas [Open Container Initiative \(OCI\)](#) con el origen y la revisión, lo que permite recuperar desde un contenedor en marcha el código exacto que ejecuta.

El registro es público, así que la máquina descarga sin credenciales y no hay ningún secreto de registro que custodiar en ella. El flujo de trabajo se autentica con el token efímero que Actions provee a cada ejecución.

9.4 Procedimiento y automatización

Un único script, `scripts/deploy.sh`, valida la configuración, descarga las imágenes, aplica la composición, espera a las comprobaciones de salud, consulta la API a través del proxy interno y muestra la revisión y el *digest* de lo que ha quedado en ejecución, junto con la dirección pública del túnel. Es idempotente: si nada ha cambiado, la descarga no trae capas nuevas y ningún contenedor se recrea.

La Figura 9.2 reproduce una ejecución completa. Merece leerse de abajo arriba: las últimas líneas imprimen la revisión del repositorio y el *digest* de cada imagen en marcha, y son las que permiten afirmar qué código exacto está sirviendo la aplicación en un momento dado. Sin ese par de datos, la única forma de responder a esa pregunta sería la confianza en que nadie tocó la máquina.

El mismo script se invoca a mano y desde una unidad de *systemd* activada por un temporizador cada cinco minutos. No se eligió esa periodicidad por necesidad operativa —no hay

```
transitops@transitops:/opt/transitops$ sudo ./scripts/deploy.sh
Descargando imágenes publicadas...
[*] Pulling 0/4
  ✓ cloudflared Pulling                                1.2s
  ✓ db Pulling                                          1.4s
  ✓ cloudflared Pulled                                2.0s
  ✓ db Pulled                                          2.0s
  ✓ api Pulled                                         2.0s
  ✓ web Pulled                                         2.0s
Aplicando el despliegue...
[*] Running 4/4
  ✓ Container transitops-db-1                          Healthy
  ✓ Container transitops-api-1                         Healthy
  ✓ Container transitops-web-1                        Healthy
  ✓ Container transitops-cloudflared-1                Healthy
Comprobando la API a través del proxy web interno...
Despliegue saludable. Estado de los contenedores:

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
transitops-api-1	ghcr.io/pey-45/transitops-api:latest	"dotnet TransitOps.A..."	api	6 minutes ago	Up 6 minutes (healthy)	8080/tcp
transitops-cloudflared-1	cloudflare/cloudflared:latest	"cloudflared --no-au..."	cloudflared	5 days ago	Up 2 hours	
transitops-db-1	postgres:16-alpine	"docker-entrypoint.s..."	db	2 hours ago	Up 2 hours (healthy)	5432/tcp
transitops-web-1	ghcr.io/pey-45/transitops-web:latest	"/docker-entrypoint..."	web	6 minutes ago	Up 6 minutes (healthy)	80/tcp

```

Imágenes desplegadas:
api: revision=82B53015e6cf97d15c50455a813333c602620e02 digest=ghcr.io/pey-45/transitops-api@sha256:7e498aa45bc24128eff3b01e244c91d0330c206ecb99ccad1c2a34d
web: revision=20283015e6cf97d15c50455a813333c602620e02 digest=ghcr.io/pey-45/transitops-web@sha256:0927b999a9b458a4cb5adcf56c12cf047c3b1d2af7e588bc44f0396
URL publica temporal: https://burns-forwarding-mortgages-clark.e.rycloudflare.com
transitops@transitops: /opt/transitops$
```

Figura 9.2: Ejecución del procedimiento: descarga, salud de los cuatro servicios, revisión y *digest* desplegados y dirección pública

tráfico que atender— sino para que la automatización sea observable dentro de una sesión de trabajo. Un ciclo sin novedades cuesta unos cinco segundos y alrededor de un segundo de CPU, coste que se registra aquí porque es la contrapartida honesta de sondear en lugar de ser notificado.

Las dos capturas siguientes documentan que esa automatización funciona sin nadie delante. La Figura 9.3 muestra el temporizador activo, con la marca de la ejecución anterior y la de la siguiente separadas por cinco minutos, que es la prueba de que el ciclo está en marcha y no simplemente instalado. La Figura 9.4 recoge el registro de una de esas ejecuciones iniciadas por *systemd*: comprueba la configuración, descarga, aplica la composición y verifica la salud igual que la invocación manual, y termina imprimiendo la misma revisión y los mismos *digests*. Esa coincidencia es el argumento de fondo del apartado, porque demuestra que la vía automática no es una variante simplificada del procedimiento documentado sino exactamente el mismo.

```
transitops@TransitOps: /opt/transitops$ systemctl list-timers
NEXT LEFT LAST PASSED UNIT ACTIVATES
Thu 2026-08-20 16:51:05 UTC 44s Thu 2026-08-20 16:46:05 UTC 4min 15s ago transitops-deploy.timer transitops-deploy.service
Thu 2026-08-20 17:39:00 UTC 48min Thu 2026-08-20 16:34:14 UTC 16min ago fwupd-refresh.timer fwupd-refresh.service
Thu 2026-08-20 18:05:15 UTC 1h 14min Sat 2026-08-15 15:17:41 UTC - apt-daily.timer apt-daily.service
Thu 2026-08-20 22:07:31 UTC 5h 17min Sun 2026-08-09 12:04:12 UTC - motd-news.timer motd-news.service
Fri 2026-08-21 00:00:00 UTC 7h Thu 2026-08-20 14:38:08 UTC 2h 12min ago dpkg-db-backup.timer dpkg-db-backup.service
Fri 2026-08-21 00:00:00 UTC 7h Thu 2026-08-20 14:38:08 UTC 2h 12min ago logrotate.timer logrotate.service
Fri 2026-08-21 00:07:00 UTC 7h - - sysstat-summary.timer sysstat-summary.service
Fri 2026-08-21 06:36:21 UTC 13h Thu 2026-08-20 14:59:18 UTC 1h 51min ago apt-daily-upgrade.timer apt-daily-upgrade.service
Fri 2026-08-21 08:21:55 UTC 15h Thu 2026-08-20 16:14:24 UTC 35min ago man-db.timer man-db.service
Fri 2026-08-21 14:43:30 UTC 21h Thu 2026-08-20 14:43:30 UTC 2h 6min ago update-notifier-download.timer update-notifier-download.service
Fri 2026-08-21 14:53:30 UTC 22h Thu 2026-08-20 14:53:30 UTC 1h 56min ago system-tmpfiles-clean.timer system-tmpfiles-clean.service
Sun 2026-08-23 03:10:51 UTC 2 days Thu 2026-08-20 13:18:16 UTC 2h 12min ago e2scrub_all.timer e2scrub_all.service
Mon 2026-08-24 01:25:55 UTC 3 days Thu 2026-08-20 14:59:46 UTC 1h 50min ago fstrim.timer fstrim.service
Tue 2026-08-25 12:12:27 UTC 5 days Sun 2026-08-09 12:04:12 UTC - update-notifier-motd.timer update-notifier-motd.service
- Thu 2026-08-20 16:50:20 UTC 21ms ago sysstat-collect.timer sysstat-collect.service

15 timers listed.
Pass --all to see loaded but inactive timers, too.
transitops@TransitOps: /opt/transitops$
```

Figura 9.3: Temporizador activo, con la ejecución anterior y la siguiente a cinco minutos de distancia

La verificación del procedimiento consistió en seguirlo desde una máquina recién creada sin desviarse del documento. Se aplicó sin correcciones: provisionar la máquina, instalar Doc-

```

Aug 20 16:46:11 TransitOps systemd[1]: transitops-deploy.service: Deactivated successfully.
Aug 20 16:46:11 TransitOps systemd[1]: Finished transitops-deploy.service - Actualizar el despliegue de TransitOps.
Aug 20 16:46:11 TransitOps systemd[1]: transitops-deploy.service: Consumed 1.127s CPU time.
Aug 20 16:51:23 TransitOps systemd[1]: Starting transitops-deploy.service - Actualizar el despliegue de TransitOps...
Aug 20 16:51:23 TransitOps deploy.sh[72136]: Descargando imágenes publicadas...
Aug 20 16:51:23 TransitOps deploy.sh[72136]: api Pulling
Aug 20 16:51:23 TransitOps deploy.sh[72136]: web Pulling
Aug 20 16:51:23 TransitOps deploy.sh[72136]: cloudflared Pulling
Aug 20 16:51:24 TransitOps deploy.sh[72136]: db Pulling
Aug 20 16:51:24 TransitOps deploy.sh[72136]: cloudflared Pulled
Aug 20 16:51:25 TransitOps deploy.sh[72136]: api Pulled
Aug 20 16:51:25 TransitOps deploy.sh[72136]: web Pulled
Aug 20 16:51:25 TransitOps deploy.sh[72136]: db Pulled
Aug 20 16:51:26 TransitOps deploy.sh[72136]: Aplicando el despliegue...
Aug 20 16:51:26 TransitOps deploy.sh[72154]: Container transitops-db-1 Running
Aug 20 16:51:26 TransitOps deploy.sh[72154]: Container transitops-api-1 Running
Aug 20 16:51:26 TransitOps deploy.sh[72154]: Container transitops-web-1 Running
Aug 20 16:51:26 TransitOps deploy.sh[72154]: Container transitops-cloudflared-1 Running
Aug 20 16:51:26 TransitOps deploy.sh[72154]: Container transitops-db-1 Waiting
Aug 20 16:51:26 TransitOps deploy.sh[72154]: Container transitops-api-1 Healthy
Aug 20 16:51:26 TransitOps deploy.sh[72154]: Container transitops-web-1 Waiting
Aug 20 16:51:26 TransitOps deploy.sh[72154]: Container transitops-api-1 Healthy
Aug 20 16:51:26 TransitOps deploy.sh[72154]: Container transitops-web-1 Healthy
Aug 20 16:51:27 TransitOps deploy.sh[72154]: Container transitops-db-1 Waiting
Aug 20 16:51:27 TransitOps deploy.sh[72154]: Container transitops-api-1 Waiting
Aug 20 16:51:27 TransitOps deploy.sh[72154]: Container transitops-web-1 Waiting
Aug 20 16:51:27 TransitOps deploy.sh[72154]: Container transitops-cloudflared-1 Waiting
Aug 20 16:51:27 TransitOps deploy.sh[72154]: Container transitops-db-1 Healthy
Aug 20 16:51:27 TransitOps deploy.sh[72154]: Container transitops-api-1 Healthy
Aug 20 16:51:27 TransitOps deploy.sh[72154]: Container transitops-web-1 Healthy
Aug 20 16:51:27 TransitOps deploy.sh[72154]: Comprobando la API a través del proxy web interno...
Aug 20 16:51:27 TransitOps deploy.sh[72154]: Despliegue saludable. Estado de los contenedores:
Aug 20 16:51:27 TransitOps deploy.sh[72246]:
Aug 20 16:51:28 TransitOps deploy.sh[72246]: NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
Aug 20 16:51:28 TransitOps deploy.sh[72246]: transitops-api-1 ghcr.io/pey-45/transitops-api:latest "dotnet TransitOps.A..." api 8 minutes ago Up 8 minutes (healthy) 8888/tcp
Aug 20 16:51:28 TransitOps deploy.sh[72246]: transitops-cloudflared-1 cloudflare/cloudflared:latest "cloudflared --no-au..." cloudflared 5 days ago Up 2 hours (healthy) 5432/tcp
Aug 20 16:51:28 TransitOps deploy.sh[72246]: transitops-db-1 postgres:16-alpine "docker-entrypoint.s..." db 2 hours ago Up 2 hours (healthy) 5432/tcp
Aug 20 16:51:28 TransitOps deploy.sh[72246]: transitops-web-1 ghcr.io/pey-45/transitops-web:latest "/docker-entrypoint..." web 8 minutes ago Up 2 minutes (healthy) 88/tcp
Aug 20 16:51:28 TransitOps deploy.sh[72886]: Imágenes desplegadas:
Aug 20 16:51:28 TransitOps deploy.sh[72886]: api: revision:8285381e5ecf97015c58045a81333c002620e02 digest:ghcr.io/pey-45/transitops-api@sha256:7e498aa45bc24128eff30e81244c91d0338c206ecb699ccad1c2a34d
Aug 20 16:51:28 TransitOps deploy.sh[72886]: web: revision:8285381e5ecf97015c58045a81333c002620e02 digest:ghcr.io/pey-45/transitops-web@sha256:0927b999a9b458a4c5adfc50c12ecf047c3b1d2af7e5880c44f8396
Aug 20 16:51:28 TransitOps deploy.sh[72886]: URL pública temporal: https://burns-forwarding-wort0ops-clahe.trycloudflare.com
Aug 20 16:51:28 TransitOps systemd[1]: transitops-deploy.service: Deactivated successfully.
Aug 20 16:51:28 TransitOps systemd[1]: Finished transitops-deploy.service - Actualizar el despliegue de TransitOps.
Aug 20 16:51:28 TransitOps systemd[1]: transitops-deploy.service: Consumed 1.118s CPU time.
transitops@TransitOps: /opt/transitops$

```

Figura 9.4: Despliegue completo iniciado por *systemd* sin intervención, con la misma revisión y los mismos *digests*

ker, clonar el repositorio, crear la configuración privada, ejecutar el script, arrancar el primer administrador y activar el temporizador. El registro del servicio muestra después ejecuciones completas iniciadas por *systemd*, con la misma revisión y los mismos *digests* que la ejecución manual, lo que confirma que la vía automática no es una variante teórica del procedimiento sino la misma.

9.5 Observabilidad mínima y límites conocidos

La monitorización es deliberadamente proporcionada: comprobaciones de salud en los cuatro contenedores, política de reinicio *unless-stopped*, y una verificación de salud en el propio script que hace fallar el despliegue si la aplicación no responde. No se incorporó recolección de métricas ni agregación de registros; para un entorno de demostración habría sido infraestructura sin lector.

Tres límites merecen quedar escritos. El primero es que el túnel rápido genera un nombre efímero: sirve para demostrar y grabar, no como alojamiento estable, y recrear el contenedor cambia la dirección. Por eso la imagen del túnel se fija por *digest*, de modo que el despliegue periódico actualice la aplicación sin mover la dirección pública. El segundo es que las pruebas de sistema validan imágenes construidas en su propio trabajo de integración, mientras que la publicación construye las que se despliegan: son construcciones distintas del mismo *commit*, y resolverlo exigiría construir una vez y publicar ese mismo artefacto. El tercero es que la base de datos desplegada es independiente de la de desarrollo, de modo que los datos de una demostración no coinciden con los locales.

Capítulo 10

Resultados

El Sprint 1 entrega el primer resultado demostrable: desde un navegador se puede introducir un usuario y contraseña, autenticar contra la API real y acceder a una zona que no existe para visitantes anónimos. El rol aparece de forma comprensible y altera la navegación. La creación del primer administrador queda separada del uso diario y se bloquea una vez cumplida su función.

El incremento también reduce riesgo futuro. El modelo de datos completo hace visibles las relaciones y restricciones antes de construir catálogos; el contrato de error evita tratamientos distintos por módulo; la migración, los contenedores y la CI establecen un camino repetible que los sprints posteriores ampliarán.

Los Sprints 2 a 4 convierten ese esqueleto en una herramienta operativa demostrable. Ya se mantienen vehículos, conductores y clientes sin perder los registros dados de baja, y se crean, consultan y editan envíos asociados a esos datos. El listado paginado y sus filtros persistentes permiten localizar trabajo por estado, fecha y recursos; la normalización UTC evita que la misma operación cambie de significado entre navegador, API y PostgreSQL.

El Sprint 4 aporta por primera vez la lógica operacional central: los recursos se asignan sin reutilizar los que ya están ocupados, una capacidad insuficiente se comunica sin impedir el trabajo y el envío avanza por un ciclo irreversible con fechas reales. La evidencia sobre PostgreSQL real confirma que las reglas no dependen del proveedor en memoria usado por las pruebas rápidas.

El Sprint 5 hace reconstruible esa operación. Creación, asignación y cambios de estado dejan una traza automática asociada a quien ejecutó la acción, mientras que puntos de control e incidencias admiten la hora real indicada por el operador. El Flujo 3 queda ya completo de extremo a extremo y todos los requisitos de prioridad alta están implementados. La línea de tiempo vacía sigue siendo válida para envíos anteriores a la migración: no se inventaron eventos retroactivos cuya fecha o actor no pudieran demostrarse.

El Sprint 6 completa la cobertura funcional RF-01–RF-14. Un administrador puede crear y

gestionar operadores sin riesgo de dejar el sistema sin administración; cualquier usuario activo puede cambiar su propia contraseña; y el Inicio resume la situación de los envíos, la actividad de recursos y las incidencias sin recuentos manuales. Los contadores enlazados convierten el resumen en una entrada al trabajo operativo, no en un informe aislado.

La aplicación quedó funcionalmente completa respecto a la línea base al cerrar el Sprint 6, de modo que el séptimo incremento se evaluó por cualidades transversales y no por alcance nuevo.

El resultado más significativo es que las limitaciones que los sprints anteriores habían declarado dejaron de existir. Las dos carreras de concurrencia —doble reserva de recursos y protección del último administrador— pasaron a estar garantizadas por el motor relacional y no solo por una comprobación en servicio, con verificación mediante operaciones simultáneas contra PostgreSQL real. La sesión dejó de exponer el token a JavaScript y de sobrevivir a una desactivación o un cambio de rol. Y las dependencias del frontend quedaron sin avisos, incluida una vulnerabilidad que afectaba a una dependencia de producción. Cada una de esas afirmaciones sustituye a un párrafo que, en el cierre del sprint correspondiente, declaraba la carencia.

El segundo resultado es que la aplicación es accesible por una dirección HTTPS pública, con los cuatro flujos de negocio recorridos y automatizados, y que el procedimiento de despliegue se siguió desde una máquina recién creada sin necesitar correcciones. La entrega quedó automatizada hasta el registro de contenedores, de forma que solo puede desplegarse un artefacto que haya superado la validación completa, y el temporizador de la máquina ejecuta el mismo procedimiento de forma autónoma. La trazabilidad entre lo desplegado y el código es explícita: el script imprime revisión y *digest* de cada imagen en ejecución.

Queda deliberadamente fuera de lo alcanzado la operación sostenida: el túnel es temporal, no hay copias de seguridad ni recolección de métricas, y el entorno es una demostración reproducible y no un servicio en producción. Distinguirlo evita atribuir al trabajo una madurez operativa que no persiguió.

Conclusiones

Los siete incrementos completados confirman la viabilidad de la arquitectura y del enfoque metodológico. El sistema cubre RF-01 a RF-14 de extremo a extremo —autenticación y arranque controlado, catálogos, envíos, asignación y ciclo de vida, trazabilidad inmutable, administración de usuarios, cambio de contraseña e indicadores operativos— y queda accesible por una dirección HTTPS pública mediante un procedimiento de despliegue reproducible. La separación SPA/API, las políticas de autorización, el modelo versionado y las pruebas automatizadas proporcionan una base mantenible sin haber ampliado el alcance más allá del núcleo necesario.

La experiencia acumulada muestra el valor de comenzar con un esqueleto transversal y añadir después rebanadas verticales. Los contratos de error, la configuración de EF en pruebas, la sesión y el proxy se validaron antes de que creciese el dominio; sobre esa base, cada regla se pudo contrastar en servicio, HTTP e interfaz.

La lección más útil, sin embargo, procede de las deudas. Cada sprint declaró por escrito lo que no había resuelto —la sesión en `localStorage`, la vigencia de los claims de un JWT ya emitido, las ventanas de concurrencia en la doble reserva y en la protección del último administrador— en lugar de presentar el incremento como completo. El endurecimiento las cerró una por una, y esa continuidad entre lo declarado y lo resuelto es más informativa que un relato sin fricción. El propio proceso lo confirmó de nuevo al final: una revisión posterior al séptimo incremento encontró trece defectos de coherencia, en su mayoría restos que el cambio de mecanismo de sesión había dejado con apariencia de estar activos. Sustituir algo transversal no termina cuando lo nuevo funciona, sino cuando lo viejo deja de aparentar que sigue en uso.

Un intercambio merece quedar enunciado sin adorno: la cookie `HttpOnly` reduce la exposición del token frente a XSS a cambio de introducir superficie CSRF, que la operación en mismo origen y `SameSite=Strict` cubren. No es una mejora gratuita, sino un cambio de riesgo elegido con conocimiento de ambos lados.

El trabajo alcanza así su objetivo sin reclamar más de lo demostrado. El entorno desplegado es una demostración reproducible y no un servicio operado: el túnel es temporal, no hay copias de seguridad ni recolección de métricas, y esa distinción se mantiene visible a lo largo de la memoria. Las líneas futuras fuera del TFG podrán incluir optimización de rutas, seguimiento por GPS, facturación o acceso de clientes externos, siempre después de validar la utilidad del núcleo operativo con uso real.

Apéndice

Trazabilidad de requisitos

Este anexo cierra la cadena que el Capítulo 4 abre. La Tabla A.1 asocia cada grupo de requisitos con el sprint que lo implementó, los componentes que lo materializan y la evidencia concreta que lo respalda. Se agrupan requisitos cuando comparten sprint y tipo de evidencia, para que la matriz siga siendo legible sin perder la propiedad que la justifica: cualquier fila puede recorrerse en sentido inverso, desde una prueba automatizada hasta la parte de la entrevista que originó el requisito. Las dos filas correspondientes a RF-04 no son un error, sino el reflejo de un requisito que se amplió en un sprint posterior al que lo introdujo.

Tabla A.1: Matriz requisito–sprint–evidencia

Requisito	Sprint	Evidencia
RF-01	S1	Login API y SPA; pruebas de credenciales, inactividad y rutas 401/403.
RF-02	S1	Endpoint protegido por token; <i>hash</i> ; conflicto con administrador activo.
RF-13	S1–S7	Contrato común, validación de modelo y componente de alerta.
RF-05–07	S2	API y SPA de vehículos, conductores y clientes; altas, edición, consulta, baja lógica y pruebas de unicidad.
RF-08, RF-12	S3	API y SPA de envíos; alta, detalle y edición; filtros persistentes, paginación y pruebas de fechas/consultas.

RF-09, RF-10	S4	API y SPA de asignación conjunta y ciclo de estados; pruebas de actividad, doble reserva, capacidad, fechas reales y terminalidad; flujo integrado sobre PostgreSQL.
RF-11	S5	API anidada y línea de tiempo inmutable; eventos automáticos y alta manual de puntos de control/incidencias; actor del JWT, orden cronológico y pruebas sobre servicio, HTTP, frontend y PostgreSQL.
RF-03, RF-04, RF-14	S6	Cambio de contraseña propia; API y SPA administrativa con política de rol y protección del último administrador; panel con estados, actividad por recurso e incidencias; pruebas de servicio, HTTP, frontend y PostgreSQL.
RF-04, RN-17	S7	Asignación de contraseña por un administrador bajo política <i>Admin</i> , con incremento de versión de sesión, sobre cuentas inactivas y con la propia excluida; pruebas de servicio, HTTP y frontend.
RNF-01– 06	S1–S7	Evidencia incremental. Validación final en S7: cuatro flujos automatizados con Playwright; cookie de sesión con <code>HttpOnly</code> , <code>Secure</code> y <code>SameSite=Strict</code> comprobada sobre HTTPS; concurrencia garantizada por el motor y verificada contra PostgreSQL real; dependencias sin avisos; y despliegue accesible reproducido siguiendo el procedimiento documentado.

Procedimientos de reproducción

Este anexo recoge los procedimientos que permiten reproducir lo que la memoria afirma haber verificado. Ninguno requiere conocimiento no escrito: si un paso necesitase una decisión que no aparece aquí, sería un defecto del procedimiento y no del lector.

Se ordenan por coste de puesta en marcha. Los dos primeros —ejecución contenerizada y verificación automatizada— solo necesitan las herramientas del proyecto y bastan para comprobar la mayor parte de lo afirmado en el Capítulo 8. El tercero exige además tener la aplicación levantada, porque las pruebas de sistema recorren la interfaz real. El cuarto reproduce el entorno accesible y es el único que requiere una máquina distinta. Cada apartado indica de forma explícita sus prerequisites, ya que la causa más habitual de que un procedimiento reproducible falle no es un error en las órdenes sino una condición previa que su autor daba por evidente.

B.1 Ejecución contenerizada

Con Docker y el complemento Compose disponibles, desde la raíz del repositorio se copia la plantilla de configuración y se levanta la composición:

```
cp .env.example .env          (copy en Windows)
docker compose up --build
```

La interfaz queda en <http://localhost:5173> y la salud de la API en <http://localhost:8080/api/v1/health>. El primer administrador se crea una sola vez mediante POST `/api/v1/auth/bootstrap-admin`, enviando usuario, correo y contraseña en JSON junto con la cabecera X-Bootstrap-Token. El fichero `.env` es obligatorio, queda ignorado por el control de versiones y sus valores son exclusivamente locales; la composición falla antes de arrancar si falta alguna variable.

B.2 Verificación automatizada

Las pruebas de servicio, de controlador y de componente no necesitan la composición levantada. Las de backend sustituyen PostgreSQL por un proveedor en memoria, salvo la clase de concurrencia, que arranca su propio contenedor mediante Testcontainers y por tanto requiere un Docker operativo:

```
dotnet restore TransitOps.slnx
dotnet build TransitOps.slnx
dotnet test TransitOps.slnx
```

```
cd frontend
npm ci
npm run lint
npm run test
npm run build
```

La validación de migraciones se realiza con `dotnet tool restore` y la opción `--migrate-only` de la API contra una instancia PostgreSQL configurada, que es la misma comprobación que ejecuta la integración continua.

B.3 Pruebas de sistema

Los cuatro flujos de negocio se ejecutan con Playwright contra la composición completa, de modo que esta debe estar levantada y respondiendo antes de lanzarlos. La preparación global arranca el primer administrador si la base todavía no lo tiene y tolera el conflicto si ya existe, y cada prueba genera sus propios datos con identificadores únicos, por lo que no hace falta reiniciar la base entre ejecuciones:

```
docker compose up --build --detach
cd frontend
npx playwright install --with-deps chromium
npm run test:e2e
```

Ante un fallo, la configuración conserva traza, captura y vídeo de la prueba afectada. La colección de la API publicada en postman/ cubre los mismos contratos de forma directa, incluida la comprobación de la cabecera `Set-Cookie` de la sesión.

B.4 Despliegue accesible

El procedimiento completo del entorno accesible —provisión de la máquina, instalación de Docker, configuración privada, ejecución del script, arranque del primer administrador y activación del temporizador— se publica en `docs/Deployment.md` y se resume en el Capítulo 9. Una vez preparada la máquina, todo el ciclo se reduce a una orden:

```
cd /opt/transitops  
sudo ./scripts/deploy.sh
```

El script valida la configuración, descarga las imágenes publicadas, aplica la composición, espera las comprobaciones de salud, verifica la API a través del proxy interno y muestra la revisión y el *digest* de lo que ha quedado en ejecución junto con la dirección pública del túnel. La misma orden la invoca el temporizador de *systemd*, de modo que la vía automática y la manual no son procedimientos distintos.

Lista de acrónimos

- API** interfaz de programación de aplicaciones. [22](#)
- CRUD** crear, consultar, actualizar y eliminar. [29](#)
- CSRF** falsificación de petición en sitios cruzados. [27](#)
- DTO** objeto de transferencia de datos. [4](#), [6](#)
- E2E** extremo a extremo. [3](#), [14](#)
- HTTP** protocolo de transferencia de hipertexto. [4](#), [22](#)
- HTTPS** protocolo de transferencia de hipertexto seguro. [57](#)
- JSON** notación de objetos de JavaScript. [6](#)
- JWT** JSON Web Token. [5](#), [25](#)
- OCI** Open Container Initiative. [58](#)
- ORM** mapeo objeto-relacional. [4](#)
- SPA** aplicación de página única. [5](#), [22](#)
- SQL** lenguaje de consulta estructurado. [4](#), [28](#)
- TLS** seguridad de la capa de transporte. [58](#)
- URL** localizador uniforme de recursos. [57](#)
- UTC** tiempo universal coordinado. [5](#), [28](#)
- UUID** identificador único universal. [5](#)
- XSS** secuencias de comandos en sitios cruzados. [27](#)

Glosario

agregado conjunto de objetos del dominio que se modifica como una unidad de consistencia. [23](#)

baja lógica desactivación de un registro sin eliminar sus datos ni sus relaciones históricas. [23](#)

doble reserva asignación simultánea de un mismo recurso a más de una operación no terminada. [5](#), [17](#), [29](#)

esqueleto andante incremento mínimo que atraviesa todas las capas de una aplicación y demuestra su integración. [9](#), [27](#), [34](#)

idempotencia propiedad por la cual repetir una operación produce el mismo estado observable que ejecutarla una sola vez. [32](#)

migración cambio versionado que transforma de forma reproducible el esquema de una base de datos. [4](#), [23](#)

máquina de estados modelo que limita los estados posibles de una entidad y las transiciones válidas entre ellos. [23](#), [30](#)

rebanada vertical incremento funcional que atraviesa interfaz, aplicación, dominio, persistencia y pruebas. [9](#)

índice único filtrado restricción de unicidad aplicada únicamente a las filas que cumplen un predicado. [5](#), [28](#), [29](#)

Bibliografía

- [1] Microsoft. (2026) Asp.net core documentation. [Online]. Available: <https://learn.microsoft.com/aspnet/core/>
- [2] ——. (2026) Entity framework core documentation. [Online]. Available: <https://learn.microsoft.com/ef/core/>
- [3] PostgreSQL Global Development Group. (2026) Postgresql documentation. [Online]. Available: <https://www.postgresql.org/docs/>
- [4] M. Jones, J. Bradley, and N. Sakimura, “Json web token (jwt),” IETF, Tech. Rep. RFC 7519, 2015.
- [5] Meta Open Source. (2026) React documentation. [Online]. Available: <https://react.dev/>
- [6] Microsoft. (2026) Typescript documentation. [Online]. Available: <https://www.typescriptlang.org/docs/>
- [7] Vite contributors. (2026) Vite guide. [Online]. Available: <https://vite.dev/guide/>
- [8] Remix Software. (2026) React router documentation. [Online]. Available: <https://reactrouter.com/>
- [9] xUnit.net contributors. (2026) xunit.net documentation. [Online]. Available: <https://xunit.net/>
- [10] Vitest contributors. (2026) Vitest guide. [Online]. Available: <https://vitest.dev/guide/>
- [11] Docker, Inc. (2026) Docker documentation. [Online]. Available: <https://docs.docker.com/>
- [12] GitHub, Inc. (2026) Github actions documentation. [Online]. Available: <https://docs.github.com/actions>
- [13] J. Humble and D. Farley, *Continuous Delivery*. Addison-Wesley, 2010.

- [14] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.
- [15] A. Cockburn, *Crystal Clear: A Human-Powered Methodology for Small Teams*. Addison-Wesley, 2004.